COSC2637 - Big Data Processing

Week 9

Introduction of Apache Spark

RMIT UNIVERSITY

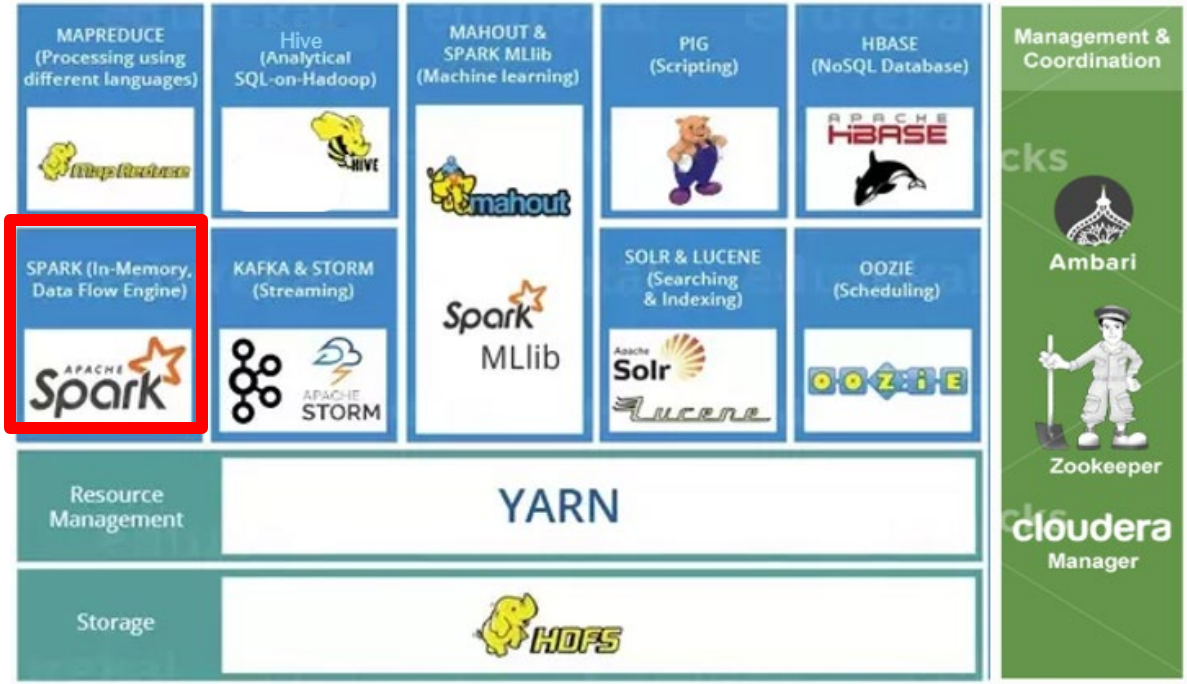
RMIT Classification: Trusted

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- Apache Spark is suitable for real-time data processing because of the processing speed.
- Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

History of Apache Spark

- 2009 - introduced in the UC Berkeley R&D Lab, i.e., AMPLab.
- 2010 - open source under BSD license.
- 2013 - donated to Apache Software Foundation
- 2014 - became top-level Apache project.



Apache Spark vs Hadoop MapReduce

Apache Spark and Hadoop MapReduce are two popular big data processing frameworks used in the industry. Both of these frameworks have the capability to handle large-scale data processing, but they differ in terms of their architecture and design.

Apache Spark vs Hadoop MapReduce

Apache Spark and Hadoop MapReduce are two popular big data processing frameworks used in the industry. Both of these frameworks have the capability to handle large-scale data processing, but they differ in terms of their architecture and design.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.

Spark is a Hadoop enhancement to MapReduce. The primary difference between Spark and MapReduce is that Spark processes and retains data in memory for subsequent steps, whereas MapReduce processes data on disk. As a result, for smaller workloads, Spark's data processing speeds are up to 100x faster than MapReduce.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.

Spark is a Hadoop enhancement to MapReduce. The primary difference between Spark and MapReduce is that Spark processes and retains data in memory for subsequent steps, whereas MapReduce processes data on disk. As a result, for smaller workloads, Spark's data processing speeds are up to 100x faster than MapReduce.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.

Spark is a Hadoop enhancement to MapReduce. The primary difference between Spark and MapReduce is that Spark processes and retains data in memory for subsequent steps, whereas MapReduce processes data on disk. As a result, for smaller workloads, Spark's data processing speeds are up to 100x faster than MapReduce.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.

Spark is a Hadoop enhancement to MapReduce. The primary difference between Spark and MapReduce is that Spark processes and retains data in memory for subsequent steps, whereas MapReduce processes data on disk. As a result, for smaller workloads, Spark's data processing speeds are up to 100x faster than MapReduce.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.

Though both platforms process data in a distributed environment, Hadoop is ideal for batch processing and linear data processing. Spark is ideal for real-time processing and processing live unstructured data streams.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.

Though both platforms process data in a distributed environment, Hadoop is ideal for batch processing and linear data processing. Spark is ideal for real-time processing and processing live unstructured data streams.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.

Though both platforms process data in a distributed environment, Hadoop is ideal for batch processing and linear data processing. Spark is ideal for real-time processing and processing live unstructured data streams.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.
3. Apache Spark works well for smaller data sets that can all fit into a server's RAM.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.
3. Apache Spark works well for smaller data sets that can all fit into a server's RAM.
4. Hadoop is more cost-effective in processing massive data sets.

Apache Spark vs Hadoop MapReduce

The Five Key Differences

1. Apache Spark is potentially 100 times faster than Hadoop MapReduce.
2. Apache Spark utilizes RAM and isn't tied to Hadoop's two-stage paradigm.
3. Apache Spark works well for smaller data sets that can all fit into a server's RAM.
4. Hadoop is more cost-effective in processing massive data sets.
5. Apache Spark is now more popular than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Spark is most effective for scenarios that involve the following:

- Achieving quick results with in-memory computations
- Analyzing stream data analysis in real time

Apache Spark vs Hadoop MapReduce

Hadoop is most effective for scenarios that involve the following:

- Processing big data sets in environments where data size exceeds available memory
- Building data analysis infrastructure with a limited budget
- Completing jobs that are not time-sensitive
- Historical and archive data analysis

5. Apache Spark is now more popular than Hadoop MapReduce.

Spark is most effective for scenarios that involve the following:

- Achieving quick results with in-memory computations
- Analyzing stream data analysis in real time

RMIT Classification: Trusted

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- Apache Spark is suitable for real-time data processing because of the processing speed.
- Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- ✓ Apache Spark is suitable for real-time data processing because of the processing speed.
- ❑ Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- ❑ Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- ❑ Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- ✓ Apache Spark is suitable for real-time data processing because of the processing speed.
- ✓ Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- ❑ Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- ❑ Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- ✓ Apache Spark is suitable for real-time data processing because of the processing speed.
- ✓ Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- ✓ Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- ❑ Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

RMIT Classification: Trusted

Q1 – Comparing Hadoop MapReduce and Apache Spark, can you claim the following:

- ✓ Apache Spark is suitable for real-time data processing because of the processing speed.
- ✓ Hadoop MapReduce is suitable in scenarios where the processing speed is not the major concern.
- ✓ Compared with Hadoop MapReduce, Apache Spark is less attractive if the data size is too large.
- ✗ Apache Spark will eventually replace Hadoop MapReduce in the big data processing market.

Q2 - Transformations are lazy means that transformations execute only when we call an action. To save space of storage, can we modify the RDD directly by reading each line and then save the changed line back to the same RDD?

- ❑ True
- ❑ False

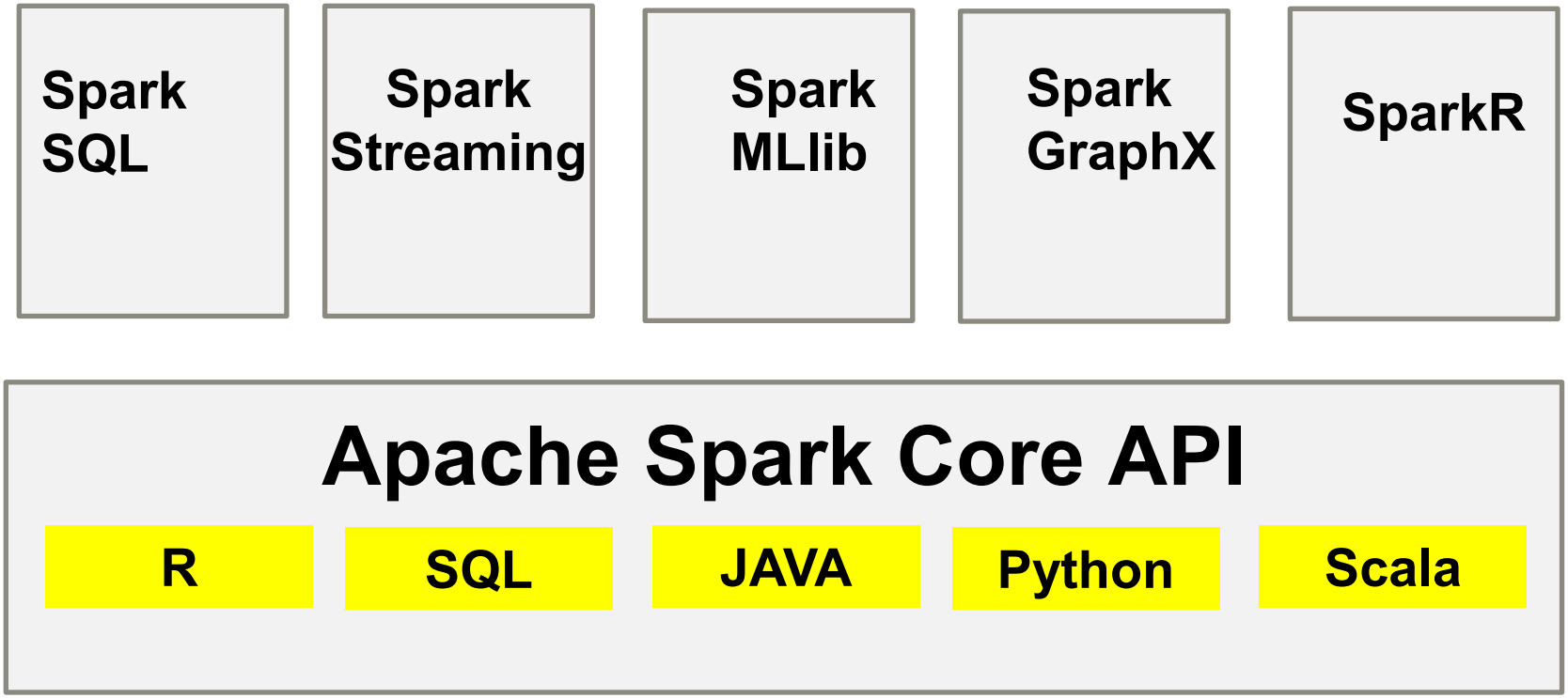
Q2 - Transformations are lazy means that transformations execute only when we call an action. To save space of storage, can we modify the RDD directly by reading each line and then save the changed line back to the same RDD?

- ❑ **True**
- ❑ **False**

Q2 - Transformations are lazy means that transformations execute only when we call an action. To save space of storage, can we modify the RDD directly by reading each line and then save the changed line back to the same RDD?

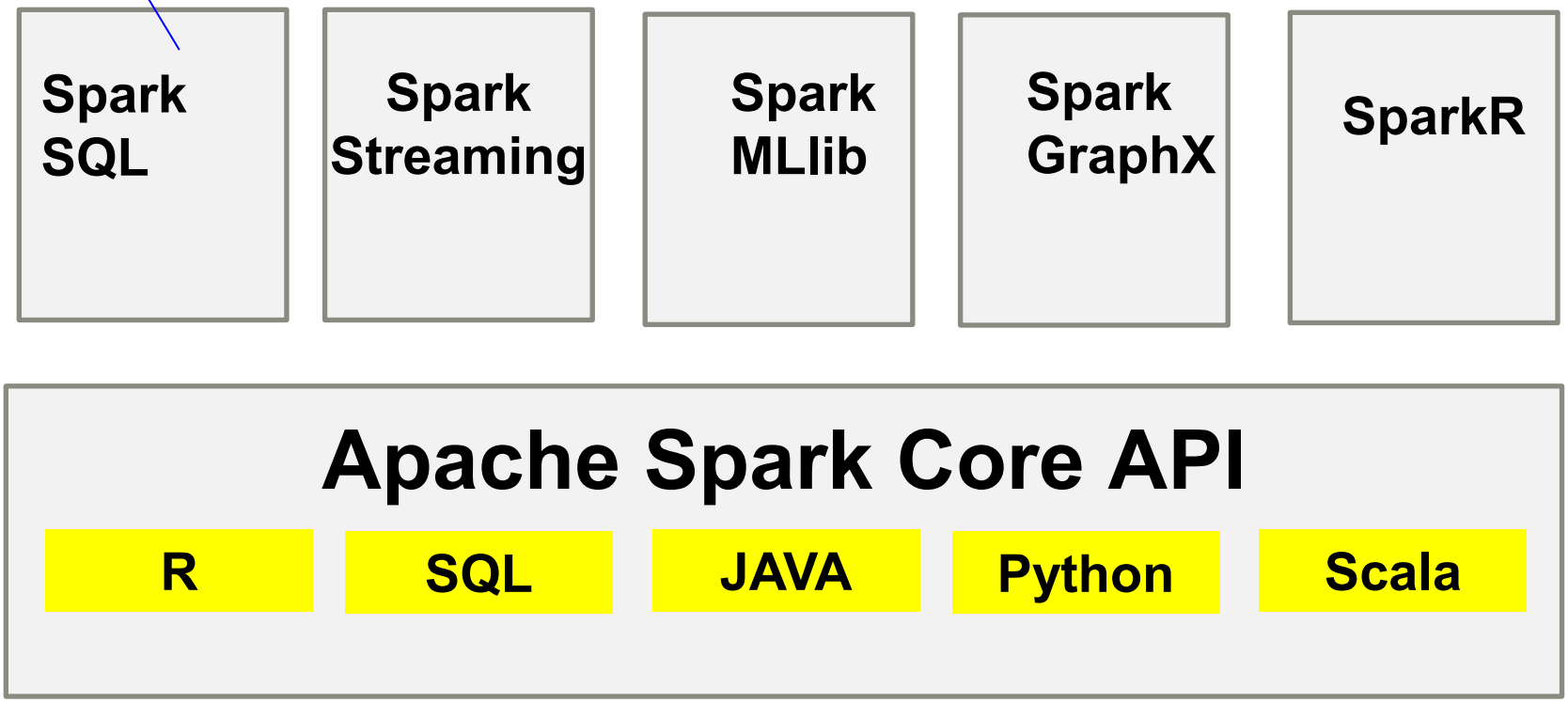
- ❑ True
- ❑ False

Apache Spark Ecosystem



Apache Spark Ecosystem

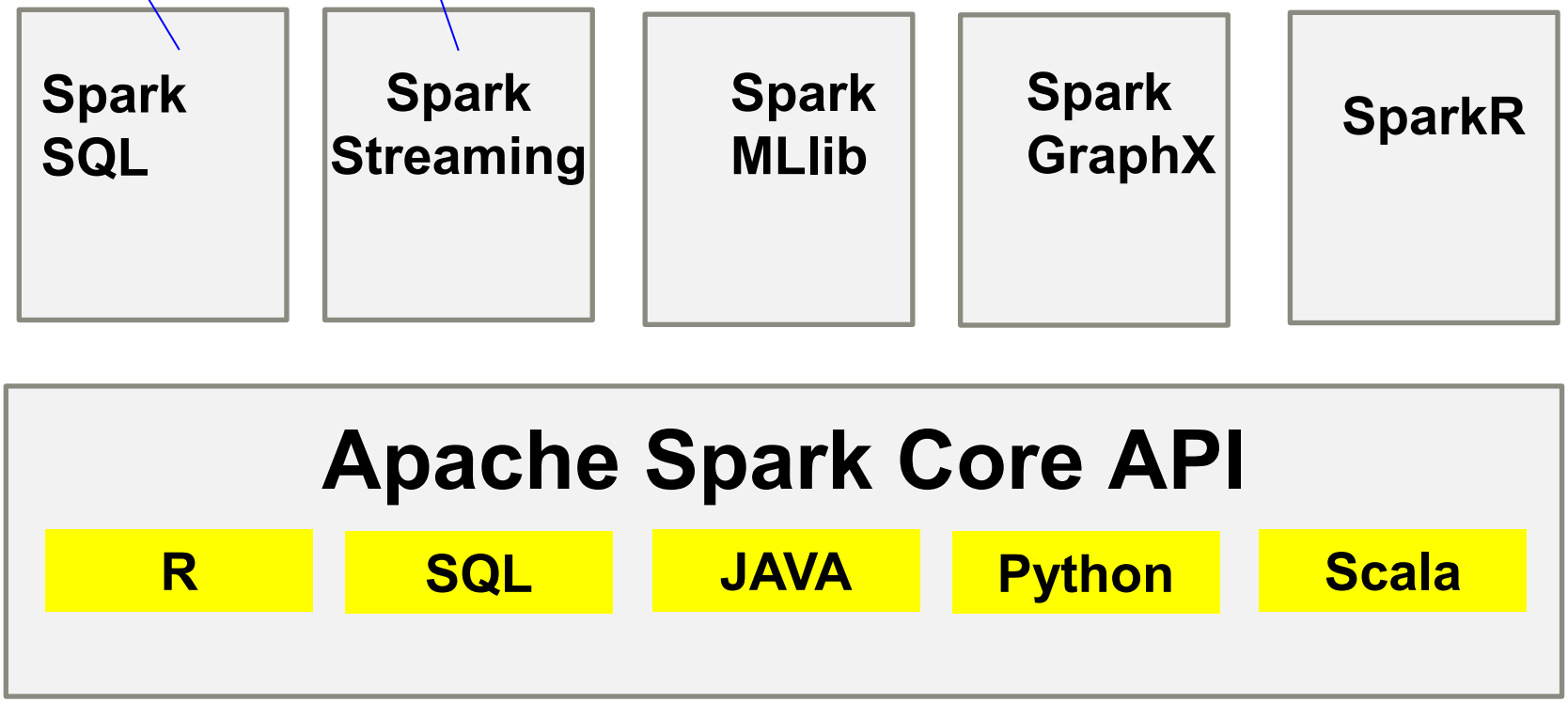
Enables users to run SQL/HQL queries on structured as well as semi-structured data



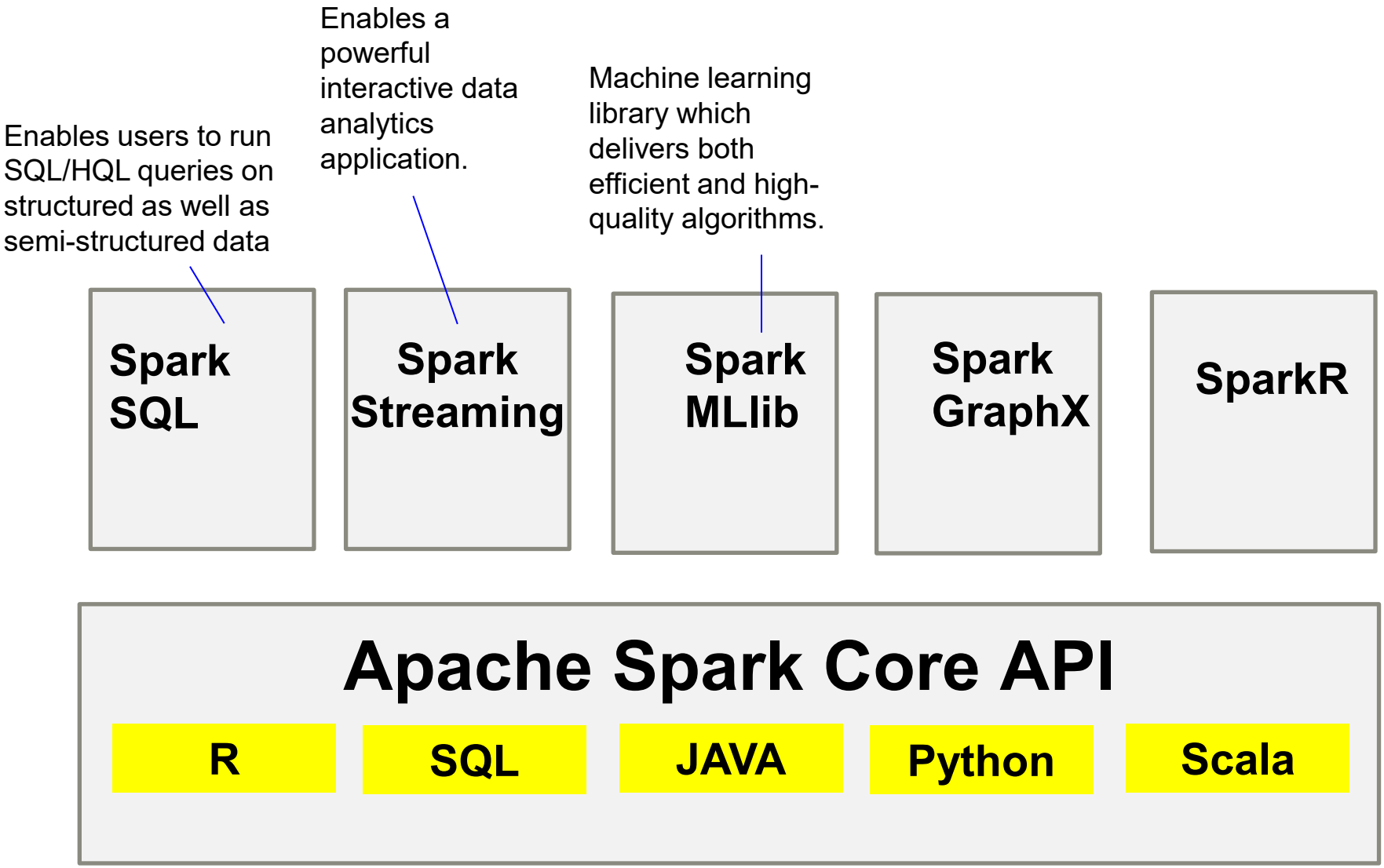
Apache Spark Ecosystem

Enables users to run SQL/HQL queries on structured as well as semi-structured data

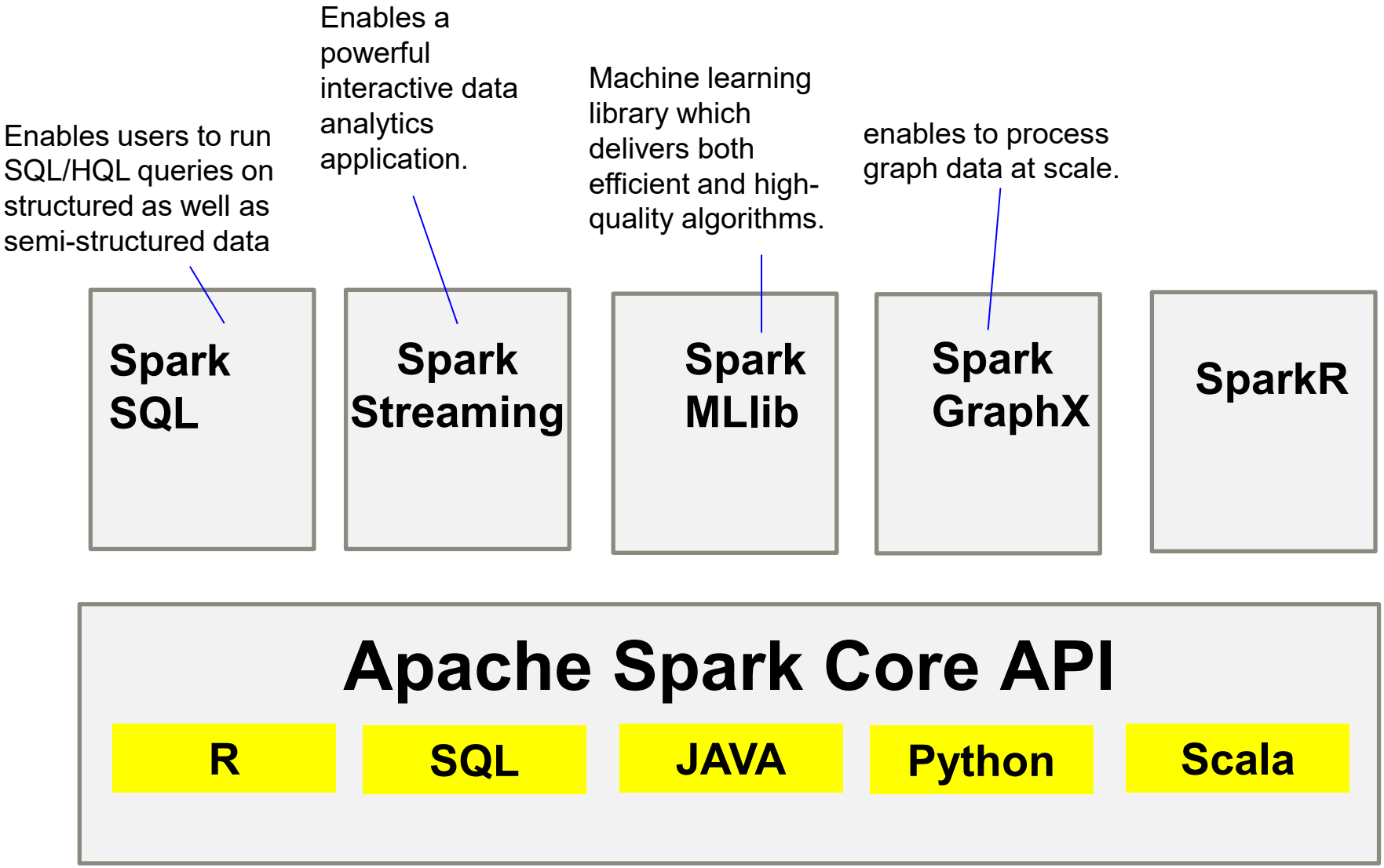
Enables a powerful interactive data analytics application.



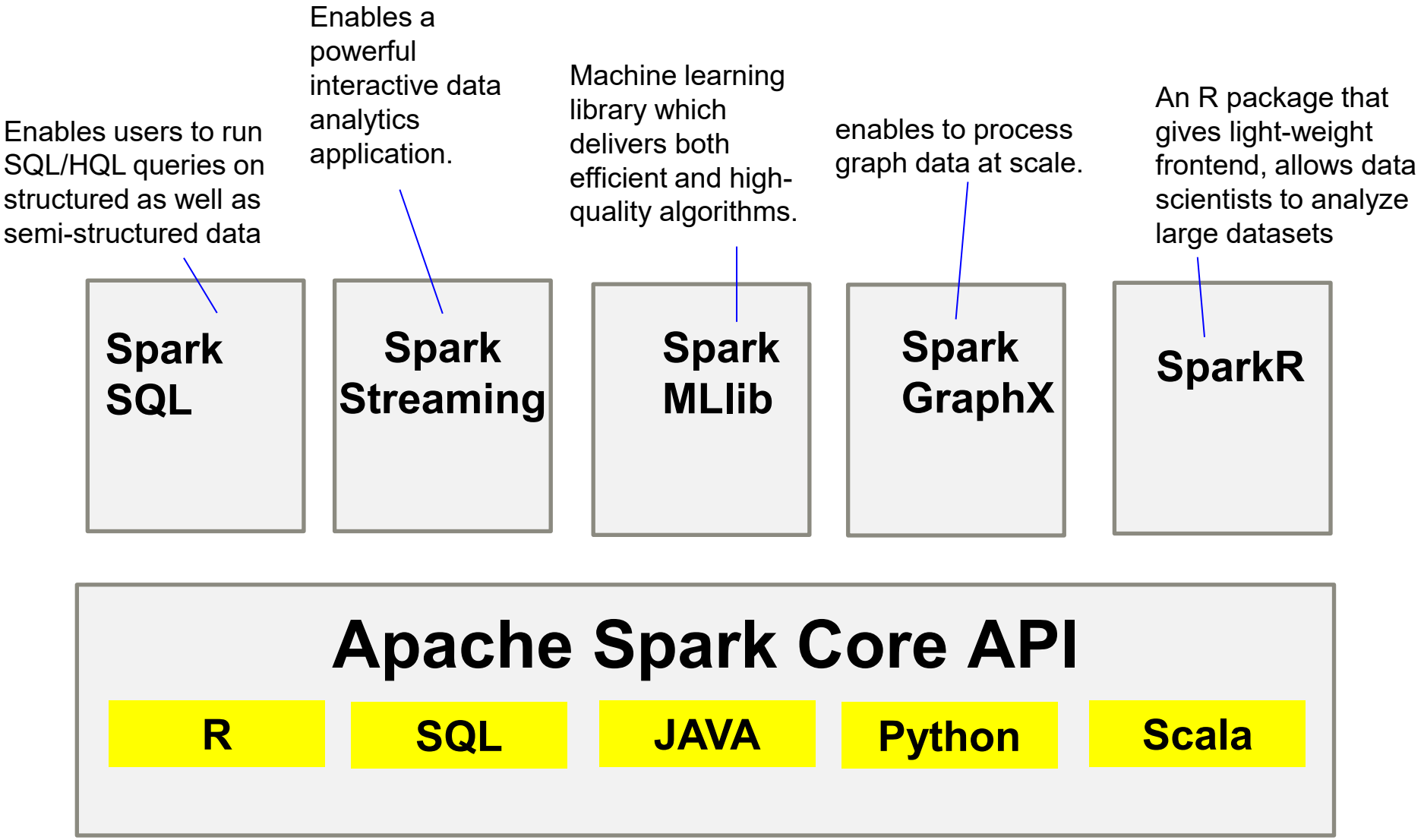
Apache Spark Ecosystem



Apache Spark Ecosystem



Apache Spark Ecosystem



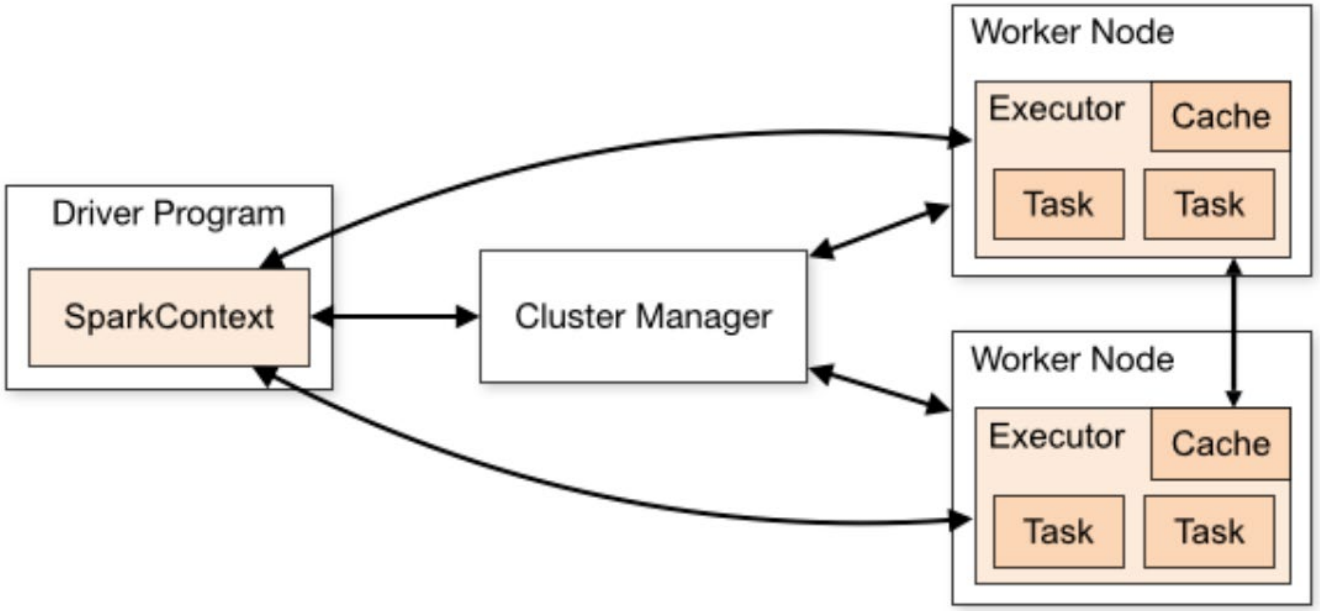
Apache Spark popular than Hadoop MapReduce

- Spark can run as a standalone application or on top of Hadoop YARN, where it can read data directly from HDFS.
  - Dozens of major tech companies are already using Spark as part of their technology stacks.
    - such as Yahoo, Intel, Baidu, Yelp, and Zillow

Apache Spark popular than Hadoop MapReduce

- Spark can run as a standalone application or on top of Hadoop YARN, where it can read data directly from HDFS.
  - Dozens of major tech companies are already using Spark as part of their technology stacks.
    - such as Yahoo, Intel, Baidu, Yelp, and Zillow

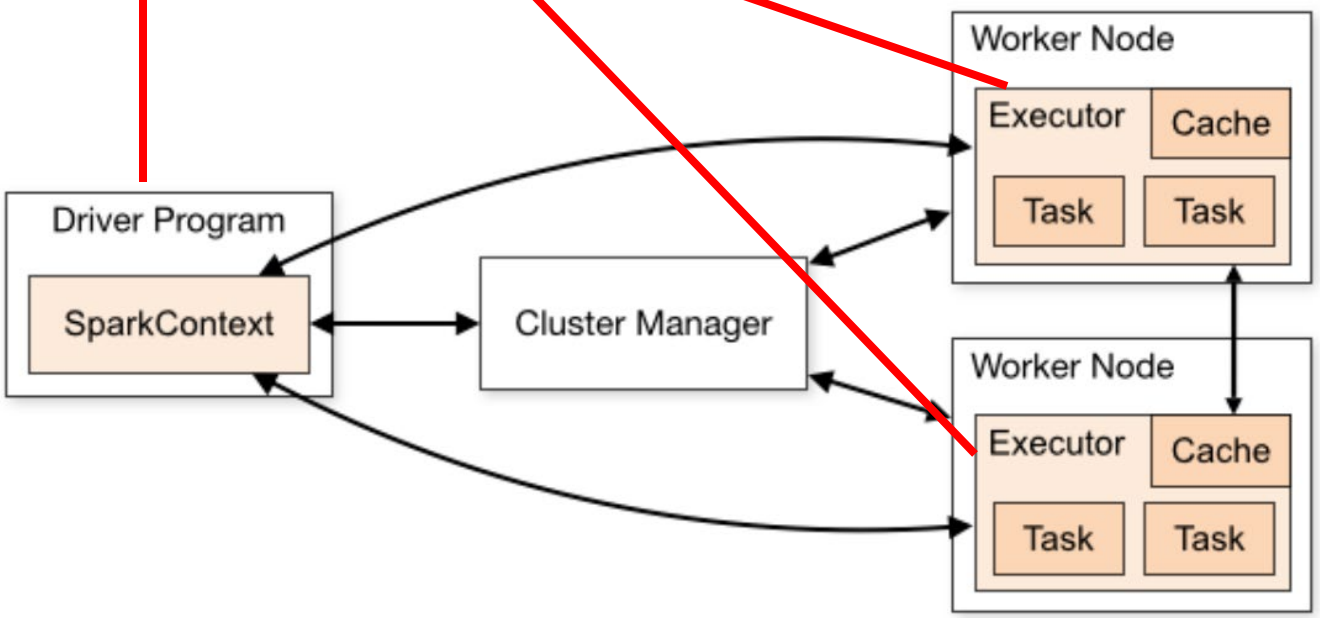
How Spark runs on clusters



How Spark runs on clusters

An application is a user program built on Spark.

- consists of a *driver program* and *executors* on the cluster.



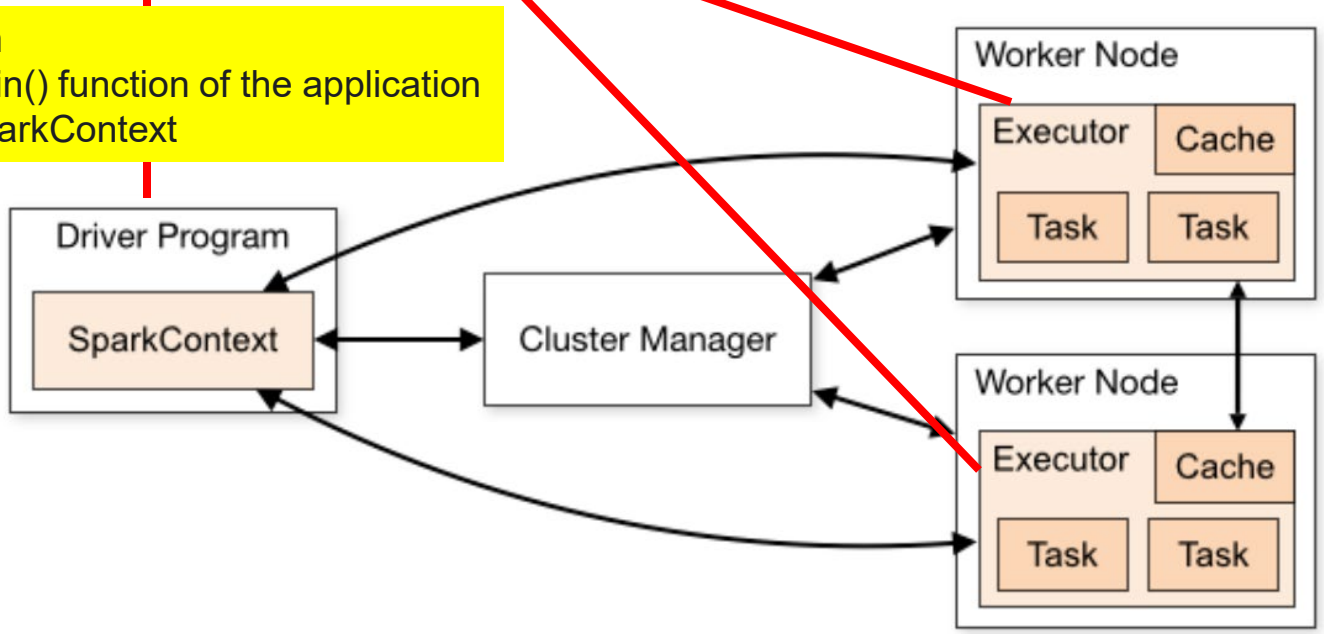
How Spark runs on clusters

An application is a user program built on Spark.

- consists of a *driver program* and *executors* on the cluster.

The driver program

- running the main() function of the application
- creating the SparkContext



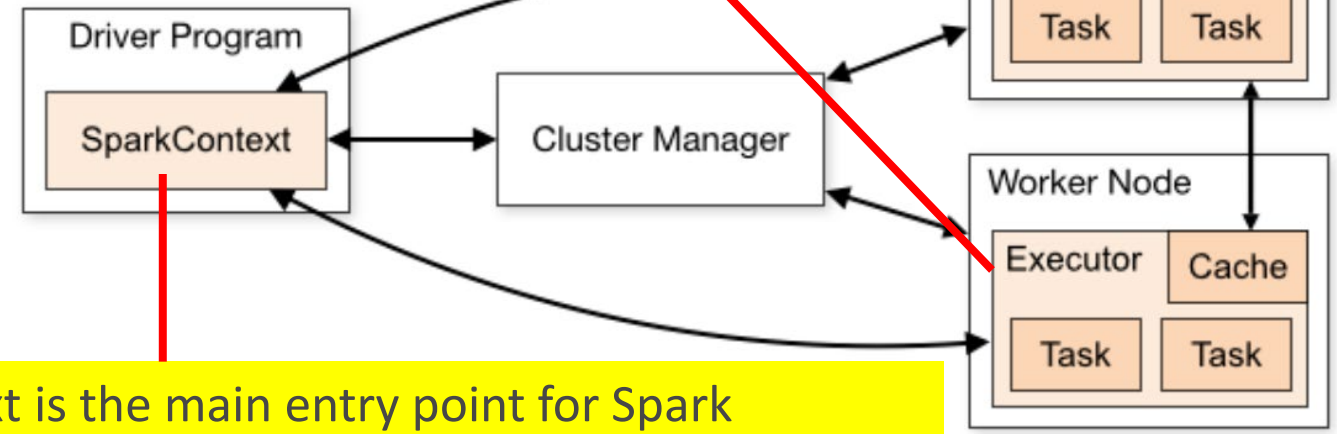
How Spark runs on clusters

An application is a user program built on Spark.

- consists of a *driver program* and *executors* on the cluster.

The driver program

- running the main() function of the application
- creating the SparkContext



SparkContext is the main entry point for Spark functionality.

- A SparkContext represents the connection to a Spark cluster, and can be used to create RDDs, accumulators and broadcast variables on that cluster.

How Spark runs on clusters

An application is a user program built on Spark.

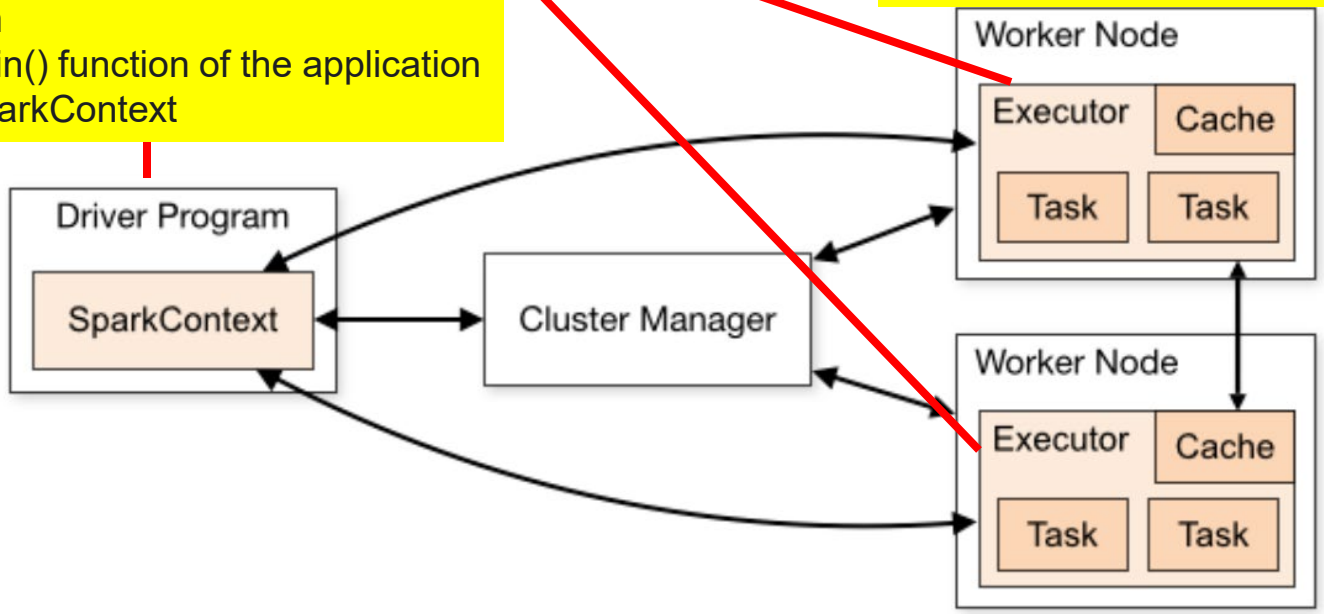
- consists of a *driver program* and *executors* on the cluster.

The driver program

- running the main() function of the application
- creating the SparkContext

An executor is a process launched for an application on a worker node,

- that runs tasks and keeps data in memory or disk storage.
- Each application has its own executors.



How Spark runs on clusters

An application is a user program built on Spark.

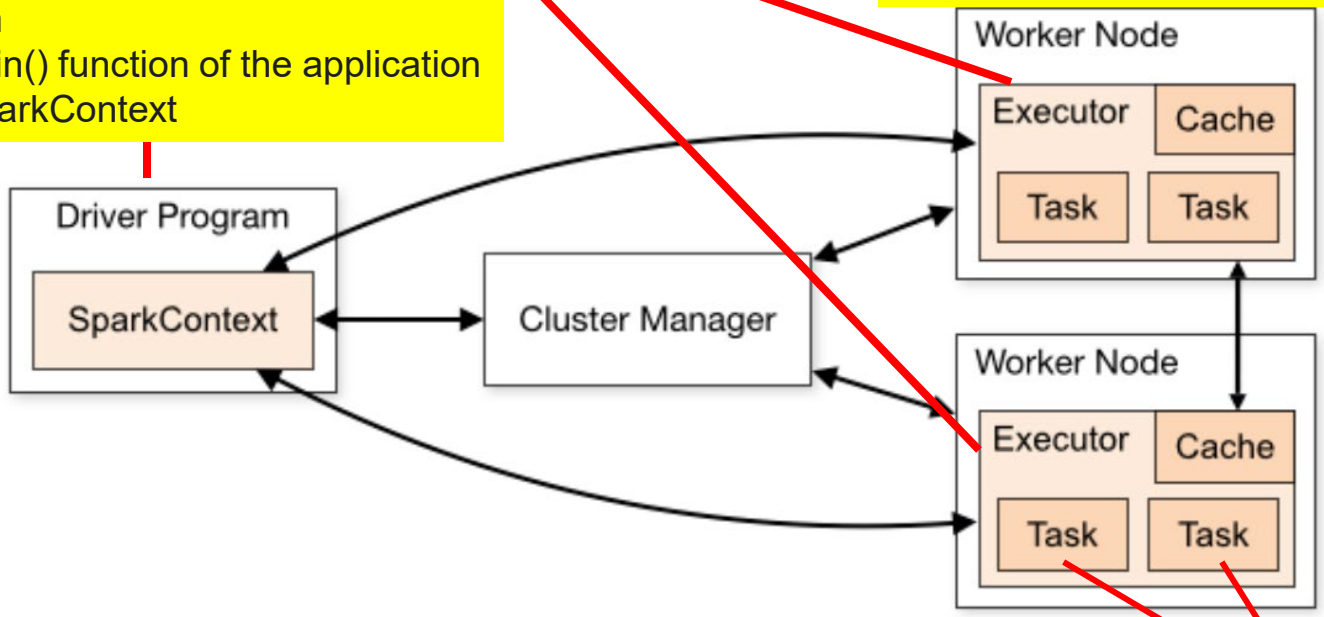
- consists of a *driver program* and *executors* on the cluster.

The driver program

- running the main() function of the application
- creating the SparkContext

An executor is a process launched for an application on a worker node,

- that runs tasks and keeps data in memory or disk storage.
- Each application has its own executors.



A task is a unit of work that will be sent to one executor

How Spark runs on clusters

An application is a user program built on Spark.

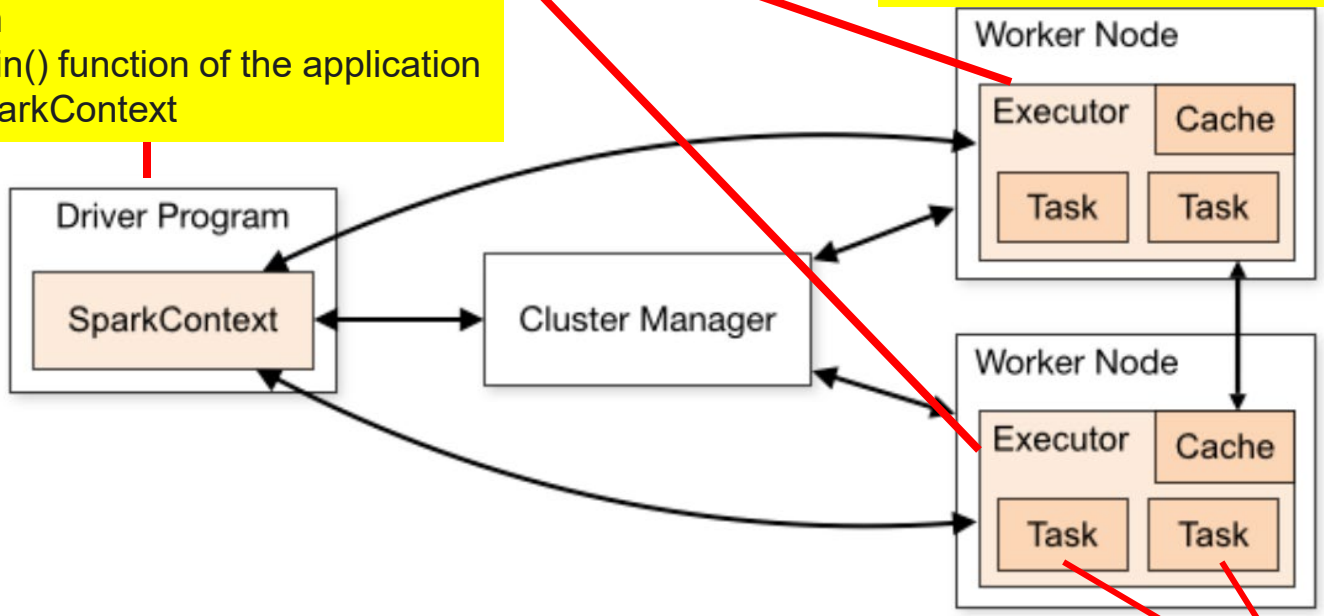
- consists of a *driver program* and *executors* on the cluster.

An executor is a process launched for an application on a worker node,

- that runs tasks and keeps data in memory or disk storage.
- Each application has its own executors.

The driver program

- running the main() function of the application
- creating the SparkContext



A job – consists of multiple tasks that gets spawned in response to a Spark action (e.g. save, collect); you'll see this term used in the driver's logs.

A task is a unit of work that will be sent to one executor

How Spark runs on clusters

An application is a user program built on Spark.

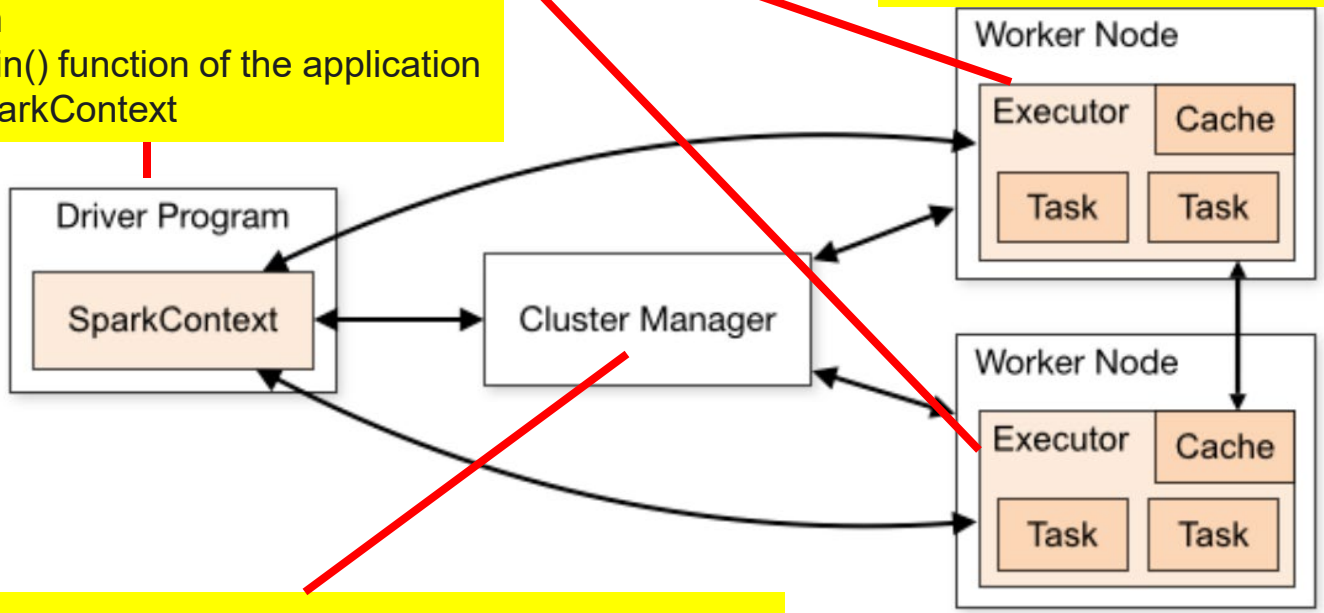
- consists of a *driver program* and *executors* on the cluster.

The driver program

- running the main() function of the application
- creating the SparkContext

An executor is a process launched for an application on a worker node,

- that runs tasks and keeps data in memory or disk storage
- Each application has its own executors.

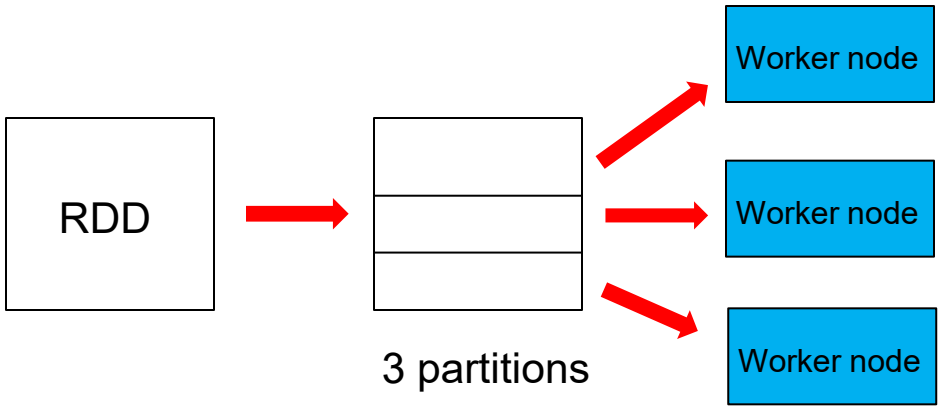


Cluster manager is an external service for acquiring resources on the cluster (e.g. standalone manager, Mesos, YARN)

Apache Spark RDD

An RDD is a collection of data items.

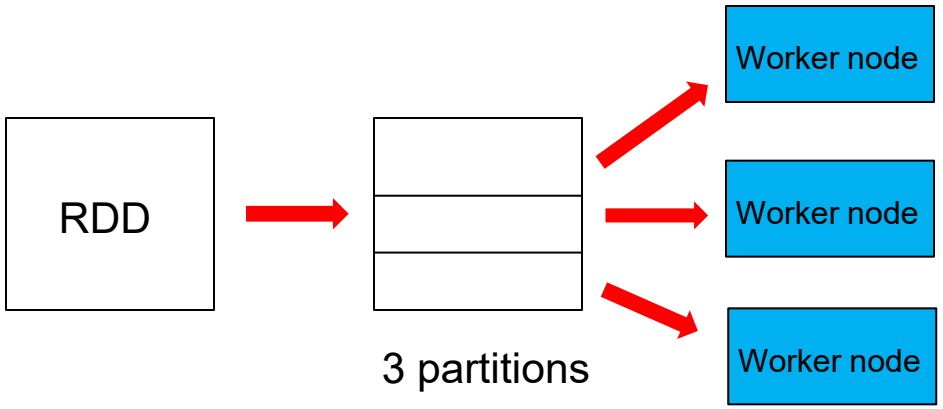
- Spark automatically partitions an RDD and distributes the partitions across different nodes.



Apache Spark RDD

An RDD is a collection of data items.

- Spark automatically partitions an RDD and distributes the partitions across different nodes.



Partitions are the basic units of parallelism in Apache Spark.

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark

There are two ways to create RDDs

- **parallelizing** an existing collection in your driver program, or
- **referencing a dataset** in an external storage system,
  - such as a shared file system, HDFS, HBase, or
  - any data source offering a Hadoop Input Format.

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark

There are two ways to create RDDs

- **parallelizing** an existing collection in your driver program, or
- **referencing a dataset** in an external storage system,
  - such as a shared file system, HDFS, HBase, or
  - any data source offering a Hadoop Input Format.

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark

There are two ways to create RDDs

- **parallelizing** an existing collection in your driver program, or
- **referencing a dataset** in an external storage system,
  - such as a shared file system, HDFS, HBase, or
  - any data source offering a Hadoop Input Format.

- RDDs are immutable (read-only) in nature. You cannot change an original RDD

Resilient Distributed Datasets (RDD) is a fundamental data structure of Spark

There are two ways to create RDDs

- **parallelizing** an existing collection in your driver program, or
- **referencing a dataset** in an external storage system,
  - such as a shared file system, HDFS, HBase, or
  - any data source offering a Hadoop Input Format.

- RDDs are immutable (read-only) in nature. You cannot change an original RDD
- but you can create new RDDs by performing operations, like transformations, on an existing RDD.

Apache Spark RDD

- Spark RDD (Resilient Distributed Dataset)
  - **Resilient**
    - Fault-tolerant with RDD lineage graph ~~(Directed Acyclic Graph)~~ and so able to recompute missing or damaged partitions due to node failures.
  - **Distributed**
    - data resides on multiple nodes.
  - **Dataset**
    - User can load the data set externally from JSON file, CSV file, text file, database via JDBC with no specific data structure.
    - parallelizing an existing collection in your driver program.

Apache Spark RDD

- Spark RDD (Resilient Distributed Dataset)
  - **Resilient**
    - Fault-tolerant with RDD lineage graph ~~(Directed Acyclic Graph)~~ and so able to recompute missing or damaged partitions due to node failures.
  - **Distributed**
    - data resides on multiple nodes.
  - **Dataset**
    - User can load the data set externally from JSON file, CSV file, text file, database via JDBC with no specific data structure.
    - parallelizing an existing collection in your driver program.

Apache Spark RDD

- Spark RDD (Resilient Distributed Dataset)
  - **Resilient**
    - Fault-tolerant with RDD lineage graph ~~(Directed Acyclic Graph)~~ and so able to recompute missing or damaged partitions due to node failures.
  - **Distributed**
    - data resides on multiple nodes.
  - **Dataset**
    - User can load the data set externally from JSON file, CSV file, text file, database via JDBC with no specific data structure.
    - parallelizing an existing collection in your driver program.

Apache Spark RDD

- Spark RDD (Resilient Distributed Dataset)
  - **Resilient**
    - Fault-tolerant with RDD lineage graph ~~(Directed Acyclic Graph)~~ and so able to recompute missing or damaged partitions due to node failures.
  - **Distributed**
    - data resides on multiple nodes.
  - **Dataset**
    - User can load the data set externally from JSON file, CSV file, text file, database via JDBC with no specific data structure.
    - parallelizing an existing collection in your driver program.

Apache Spark RDD

- Spark RDD (Resilient Distributed Dataset)
  - **Resilient**
    - Fault-tolerant with RDD lineage graph (~~Directed Acyclic Graph~~) and so able to recompute missing or damaged partitions due to node failures.
  - **Distributed**
    - data resides on multiple nodes.
  - **Dataset**
    - User can load the data set externally from JSON file, CSV file, text file, database via JDBC with no specific data structure.
    - parallelizing an existing collection in your driver program.

Apache Spark RDD Operations

Types of RDD Operations

- Transformations
- Actions

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - “Applying transformations” built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - "Applying transformations" built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - "Applying transformations" built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - "Applying transformations" built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - "Applying transformations" built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - They are not executed immediately.
  - Transformations execute only when we call an action.
- RDD lineage
  - "Applying transformations" built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

Apache Spark RDD Operations

RDD Transformation

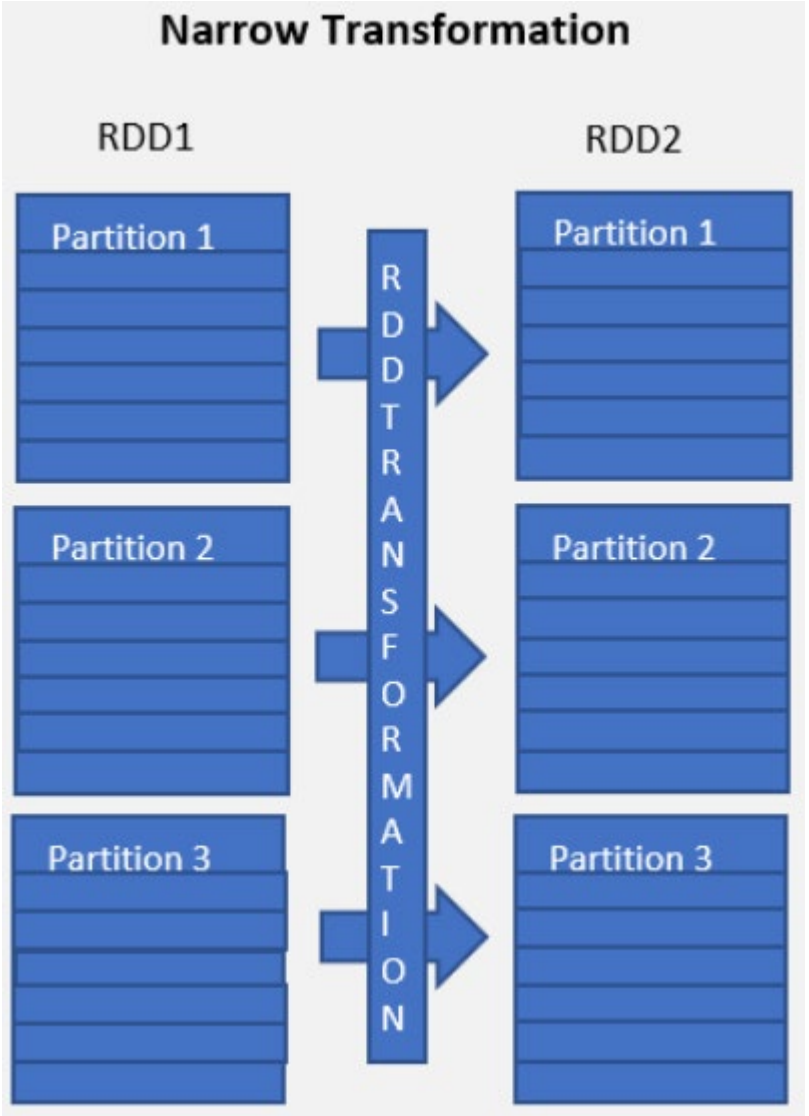
- Two types of transformations
  - **Narrow transformation**

    All the elements that are required to compute the records in a single partition live in the single partition of parent RDD.

    e.g., *map(), filter().*
  - **Wide transformation**

    All the elements that are required to compute the records in the single partition may live in many partitions of parent RDD.

    e.g., *groupByKey(), aggregateByKey(), aggregate(), join(), repartition()*



https://sauravomar01.medium.com/wide-vs-narrow-dependencies-in-apache-spark-2cd33bf7ed7d

Apache Spark RDD Operations

RDD Transformation

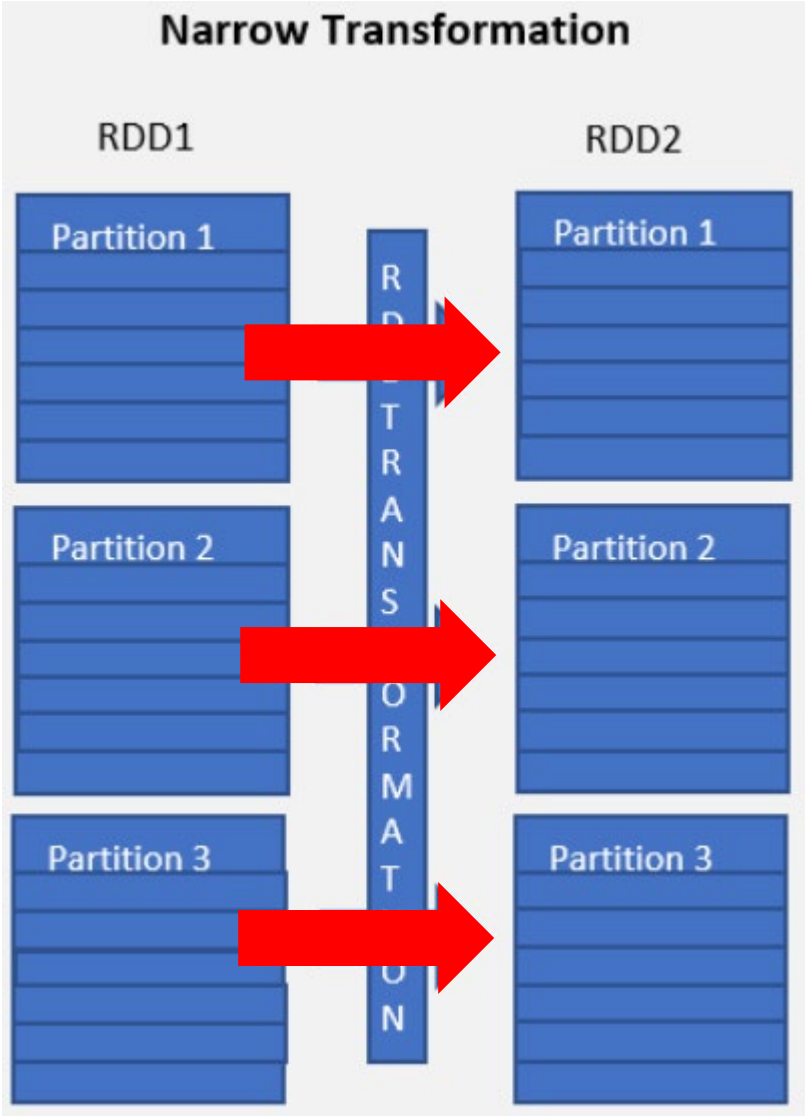
- Two types of transformations
  - **Narrow transformation**

    All the elements that are required to compute the records in a single partition live in the single partition of parent RDD.

    e.g., *map(), filter().*
  - **Wide transformation**

    All the elements that are required to compute the records in the single partition may live in many partitions of parent RDD.

    e.g., *groupByKey(), aggregateByKey(), aggregate(), join(), repartition()*



https://sauravomar01.medium.com/wide-vs-narrow-dependencies-in-apache-spark-2cd33bf7ed7d

Apache Spark RDD Operations

RDD Transformation

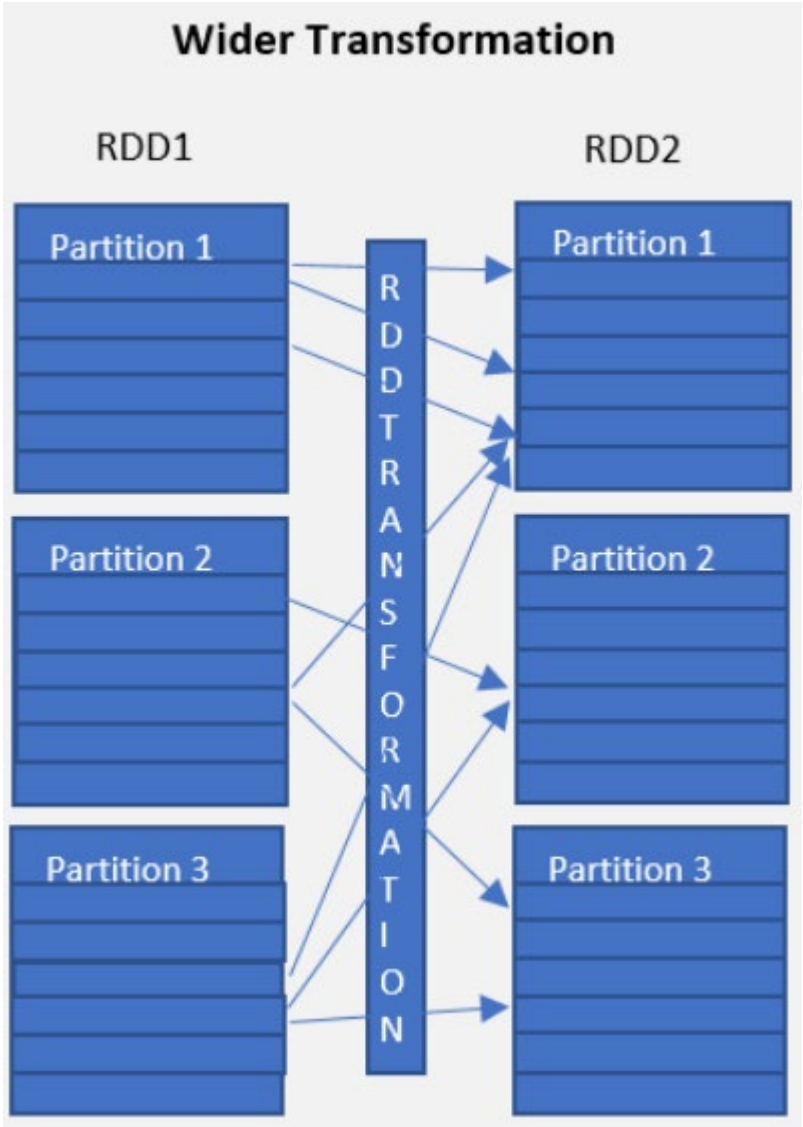
- Two types of transformations
  - **Narrow transformation**

    All the elements that are required to compute the records in a single partition live in the single partition of parent RDD.

    e.g., *map(), filter().*
  - **Wide transformation**

    All the elements that are required to compute the records in the single partition may live in many partitions of parent RDD.

    e.g., *groupByKey(), aggregateByKey(), aggregate(), join(), repartition()*



https://sauravomar01.medium.com/wide-vs-narrow-dependencies-in-apache-spark-2cd33bf7ed7d

Apache Spark RDD Operations

RDD Transformation

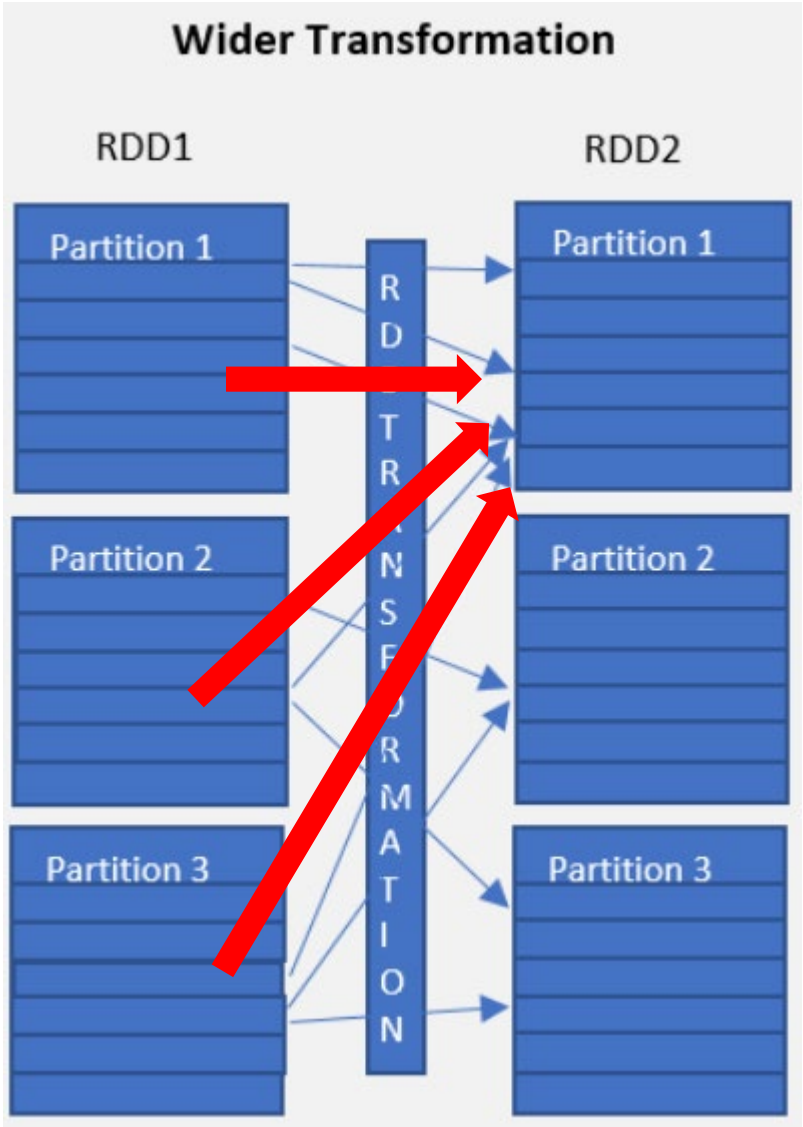
- Two types of transformations
  - **Narrow transformation**

    All the elements that are required to compute the records in a single partition live in the single partition of parent RDD.

    e.g., *map(), filter().*
  - **Wide transformation**

    All the elements that are required to compute the records in the single partition may live in many partitions of parent RDD.

    e.g., *groupByKey(), aggregateByKey(), aggregate(), join(), repartition()*



https://sauravomar01.medium.com/wide-vs-narrow-dependencies-in-apache-spark-2cd33bf7ed7d

Apache Spark RDD Operations

RDD Transformation

- map()
  - The map function iterates over every line in RDD and split into new RDD.
  - Using **map()** transformation, we take in any function and that function is applied to every element of RDD.

For example, RDD {1, 2, 3, 4, 5}

```
rdd.map(x=>x+2)
```

we will get the result as (3, 4, 5, 6, 7).

Apache Spark RDD Operations

RDD Transformation – some examples

- map()
  - The map function iterates over every line in RDD and split into new RDD.
  - Using **map()** transformation, we take in any function and that function is applied to every element of RDD.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.       val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.       val data = spark.read.textFile("spark_test.txt").rdd
7.       val mapFile = data.map(line => (line, line.length))
8.       mapFile.foreach(println)
9.    }
8. }
```

each line is an element

In map() transformation, the length for each line in the file is calculated. For example:

pigs -> pigs, 4
dog -> dog, 3
My dog -> My dog, 6

Apache Spark RDD Operations

RDD Transformation – some examples

- flatMap()
  - The most simple use of flatMap() is to split each input string into words.
  - Map and flatMap are similar in the way that they take a line from input RDD and apply a function on that line.

Apache Spark RDD Operations

RDD Transformation – some examples

- flatMap()
  - The most simple use of flatMap() is to split each input string into words.
  - Map and flatMap are similar in the way that they take a line from input RDD and apply a function on that line.

Apache Spark RDD Operations

RDD Transformation – some examples

- flatMap()
  - The most simple use of flatMap() is to split each input string into words.
  - Map and flatMap are similar in the way that they take a line from input RDD and apply a function on that line.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.      val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.        val data = spark.read.textFile("spark_test.txt").rdd
7.        val flatmapFile = data.flatMap(lines => lines.split(","))
8.        flatmapFile.foreach(println)
9.    }
8. }
```

flatMap() function splits each line when comma occurs.

Given two elements [1,2] [1,1]

Map - [1,2],[1,1] => [[1,2],[1,1]]

vs.

flatMap - [1,2],[1,1] => [1,2,1,1]

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

For example, RDD contains (1, 2, 3, 4, 5) and the predicate check for an even number.

- The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

For example, RDD contains (1, 2, 3, 4, 5) and the predicate check for an even number.
- The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.       val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.       val data = spark.read.textFile("spark_test.txt").rdd
         val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
         println(mapFile.count())
3.    }
8. }
```

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

For example, RDD contains (1, 2, 3, 4, 5) and the predicate check for an even number.
- The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.       val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.       val data = spark.read.textFile("spark_test.txt").rdd
         val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
         println(mapFile.count())
3.    }
8. }
```

flatMap function maps each line into words

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

For example, RDD contains (1, 2, 3, 4, 5) and the predicate check for an even number.
- The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.      val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.      val data = spark.read.textFile("spark_test.txt").rdd
        val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
        println(mapFile.count())
3.    }
8. }
```

flatMap function maps each line into words

Then use filter such that only word "spark" is retained

Apache Spark RDD Operations

RDD Transformation – some examples

- filter(func)
  - returns a new RDD, containing only the elements that meet a predicate.
  - It is a *narrow operation* - it does not shuffle data from one partition to many partitions.

For example, RDD contains (1, 2, 3, 4, 5) and the predicate check for an even number.
- The resulting RDD after the filter will contain only the even numbers i.e., 2 and 4.

```
1. import org.apache.spark.SparkContext
2. import org.apache.spark.SparkConf
3. import org.apache.spark.sql.SparkSession

3.  object mapTest{
4.    def main(args: Array[String]) = {
5.      val spark = SparkSession.builder.appName("mapExample").master("local").getOrCreate()
6.      val data = spark.read.textFile("spark_test.txt").rdd
        val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
        println(mapFile.count())
3.    }
8. }
```

flatMap function maps each line into words

Then use filter such that only word "spark" is retained

Then count the word retained

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation – some examples

- union(dataset)

  For example,
  - RDD1 are (Spark, Spark, Hadoop, Flink)
  - RDD2 are (Big data, Spark, Flink)

The resultant *rdd1.union(rdd2)* have elements
(Spark, Spark, Spark, Hadoop, Flink, Flink, Big data).

Apache Spark RDD Operations

RDD Transformation – some examples

- union(dataset)

  For example,
  - RDD1 are (Spark, Spark, Hadoop, Flink)
  - RDD2 are (Big data, Spark, Flink)

  The resultant *rdd1.union(rdd2)* have elements (Spark, Spark, Spark, Hadoop, Flink, Flink, Big data).

Apache Spark RDD Operations

RDD Transformation – some examples

- union(dataset)

  For example,
  - RDD1 are (Spark, Spark, Hadoop, Flink)
  - RDD2 are (Big data, Spark, Flink)

  The resultant *rdd1.union(rdd2)* have elements (Spark, Spark, Spark, Hadoop, Flink, Flink, Big data).

```
1. val rdd1 = spark.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014),(16,"feb",2014)))
2. val rdd2 = spark.sparkContext.parallelize(Seq((5,"dec",2014),(17,"sep",2015)))
3. val rdd3 = spark.sparkContext.parallelize(Seq((6,"dec",2011),(16,"may",2015)))
4. val rddUnion = rdd1.union(rdd2).union(rdd3)
5. rddUnion.foreach(println)
```

Apache Spark RDD Operations

RDD Transformation – some examples

- union(dataset)

  For example,
  - RDD1 are (Spark, Spark, Hadoop, Flink)
  - RDD2 are (Big data, Spark, Flink)

  The resultant *rdd1.union(rdd2)* have elements (Spark, Spark, Spark, Hadoop, Flink, Flink, Big data).

```
1. val rdd1 = spark.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014),(16,"feb",2014)))
2. val rdd2 = spark.sparkContext.parallelize(Seq((5,"dec",2014),(17,"sep",2015)))
3. val rdd3 = spark.sparkContext.parallelize(Seq((6,"dec",2011),(16,"may",2015)))
4. val rddUnion = rdd1.union(rdd2).union(rdd3)
5. rddUnion.foreach(println)
```

(1,"jan",2016),(3,"nov",2014),(16,"feb",2014),(5,"dec",2014),
(17,"sep",2015) (6,"dec",2011),(16,"may",2015)

Apache Spark RDD Operations

RDD Transformation – some examples

- intersection(other-dataset)

  For example

  - **RDD1** are (Spark, Spark, Hadoop, Flink)
  - **RDD2** are (Big data, Spark, Flink)

Apache Spark RDD Operations

RDD Transformation – some examples

- intersection(other-dataset)

  For example

  - **RDD1** are (Spark, Spark, Hadoop, Flink)
  - **RDD2** are (Big data, Spark, Flink)

The resultant rdd1.intersection(rdd2) will have elements (Spark, Flink).

Apache Spark RDD Operations

RDD Transformation – some examples

- intersection(other-dataset)

  For example

  - **RDD1** are (Spark, Spark, Hadoop, Flink)
  - **RDD2** are (Big data, Spark, Flink)

The resultant rdd1.intersection(rdd2) will have elements (Spark, Flink).

```
1. val rdd1 = spark.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014), (16,"feb",2014)))
2. val rdd2 = spark.sparkContext.parallelize(Seq((5,"dec",2014),(1,"jan",2016)))
3. val comman = rdd1.intersection(rdd2)
4. comman.foreach(println)
```

Apache Spark RDD Operations

RDD Transformation – some examples

- intersection(other-dataset)

  For example

  - **RDD1** are (Spark, Spark, Hadoop, Flink)
  - **RDD2** are (Big data, Spark, Flink)

The resultant rdd1.intersection(rdd2) will have elements (Spark, Flink).

```
1. val rdd1 = spark.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014), (16,"feb",2014)))
2. val rdd2 = spark.sparkContext.parallelize(Seq((5,"dec",2014),(1,"jan",2016)))
3. val comman = rdd1.intersection(rdd2)
4. comman.foreach(println)
```

(1,"jan",2016)

Apache Spark RDD Operations

RDD Transformation – some examples

- distinct()

It returns a new dataset that contains the **distinct** elements of the source dataset. It is helpful to remove duplicate data. For example,

- RDD has elements (Spark, Spark, Hadoop, Flink)
- *rdd.distinct()* will give elements (Spark, Hadoop, Flink).

Apache Spark RDD Operations

RDD Transformation – some examples

- distinct()

It returns a new dataset that contains the **distinct** elements of the source dataset. It is helpful to remove duplicate data. For example,

- RDD has elements (Spark, Spark, Hadoop, Flink)
- *rdd.distinct()* will give elements (Spark, Hadoop, Flink).

Apache Spark RDD Operations

RDD Transformation – some examples

- distinct()

It returns a new dataset that contains the **distinct** elements of the source dataset. It is helpful to remove duplicate data. For example,

- RDD has elements (Spark, Spark, Hadoop, Flink)
- *rdd.distinct()* will give elements (Spark, Hadoop, Flink).

```
val rdd1 =  park.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014),(3,"nov",2014)))
val result = rdd1.distinct()
println(result.collect().mkString(", "))
```

Apache Spark RDD Operations

RDD Transformation – some examples

- distinct()

It returns a new dataset that contains the **distinct** elements of the source dataset. It is helpful to remove duplicate data. For example,

- RDD has elements (Spark, Spark, Hadoop, Flink)
- *rdd.distinct()* will give elements (Spark, Hadoop, Flink).

```
val rdd1 =   park.sparkContext.parallelize(Seq((1,"jan",2016),(3,"nov",2014),(3,"nov",2014)))
val result = rdd1.distinct()
println(result.collect().mkString(", "))
```

the distinct function will remove the duplicate elements (3,'"nov",2014).

Apache Spark RDD Operations

RDD Transformation – some examples

- groupByKey()

Group the values for each key in the RDD into a single sequence. Splitting the resultant RDD into a number of partitions (numPartitions).

```
val rdd1 = sc.parallelize(Seq((‘a’, 1), (‘b’, 1), (‘a’, 1)))
rdd1.groupByKey().sortByKey().collect()
```

[('a’, [1,1]), ('b’, [1])]

Apache Spark RDD Operations

RDD Transformation – some examples

- groupByKey()

Group the values for each key in the RDD into a single sequence. Splitting the resultant RDD into a number of partitions (numPartitions).

```
val rdd1 = sc.parallelize(Seq((‘a’, 1), (‘b’, 1), (‘a’, 1)))
rdd1.groupByKey().sortByKey().collect()
```

[('a’, [1,1]), ('b’, [1])]

```
rdd1.groupByKey().mapValues(x=>x.size).sortByKey().collect()
```

[('a', 2), ('b', 1)]

mapValues(function) - pass each value in the key-value pair RDD through a map function without changing the keys
e.g., [1,1] for key ‘a’

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

parallelize RDD (the Array of String) – create RDD from an array

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

parallelize RDD (the Array of String) – create RDD from an array

It will then map each word with count 1
(one, 1)
(two, 1)
(two, 1)
…

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

parallelize RDD (the Array of String) – create RDD from an array

It will then map each word with count 1
(one, 1)
(two, 1)
(two, 1)
…

then reduceByKey will aggregate values for the same key.

Apache Spark RDD Operations

RDD Transformation – some examples

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

parallelize RDD (the Array of String) – create RDD from an array

It will then map each word with count 1
(one, 1)
(two, 1)
(two, 1)
…

then reduceByKey will aggregate values for the same key.
(one, 1)
(two, 2)
…

Apache Spark RDD

RDD Transformation – some examples

This will also perform local aggregation on each mapper before sending results to a reducer, similarly to a "combiner" in MapReduce.

- reduceByKey()

Using **reduceByKey** on a dataset (K, V), the pairs on the same machine with the same key are combined, before the data is shuffled.

```
val words = Array("one","two","two","four","five","six","six","eight","nine","ten")
val data = spark.sparkContext.parallelize(words).map(w => (w,1)).reduceByKey(_+_)
data.foreach(println)
```

parallelize RDD (the Array of String) – create RDD from an array

It will then map each word with count 1
(one, 1)
(two, 1)
(two, 1)
…

then reduceByKey will merge values for the same key.
(one, 1)
(two, 2)
…

Apache Spark RDD Operations

Types of RDD Operations

- Transformations
- Actions

Apache Spark RDD Operations
RDD Action

Actions are Spark RDD operations that give non-RDD values.

- The values returned by action are stored to the driver or external storage.
- It brings laziness of transformations into motion.

Apache Spark RDD Operations
RDD Action

Actions are Spark RDD operations that give non-RDD values.

- The values returned by action are stored to the driver or external storage.
- It brings laziness of transformations into motion.

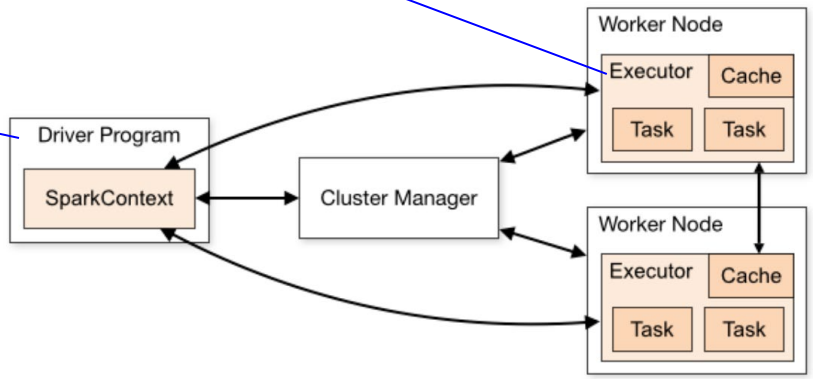
sending data from *executers* to the *driver*

Apache Spark RDD Operations
RDD Action

Actions are Spark RDD operations that give non-RDD values.

- The values returned by action are stored to the driver or external storage.
- It brings laziness of transformations into motion.

sending data from *executers* to the *driver*



Apache Spark RDD Operations
RDD Action

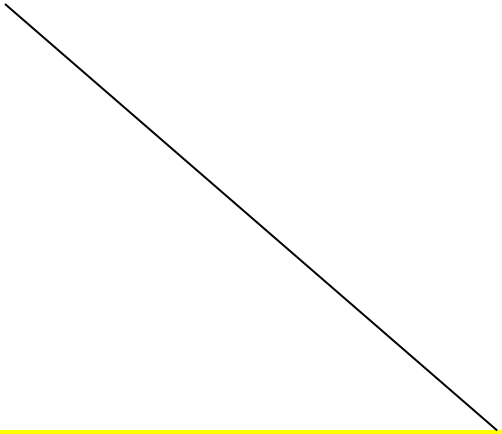
Actions are Spark RDD operations that give non-RDD values.

- The values returned by action are stored to the driver or external storage.
- It brings laziness of transformations into motion.

Apache Spark RDD Operations
RDD Action

Actions are Spark RDD operations that give non-RDD values.

- The values returned by action are stored to the driver or external storage.
- It brings laziness of transformations into motion.



Transformations are lazy

- Transformations execute only when we call an action.
- They are not executed immediately.

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6}
"rdd.count()" will give the result 8.

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6}
"rdd.count()" will give the result 8.

Create an RDD by loading a file from storage system

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6}
"rdd.count()" will give the result 8.

Create an RDD by loading a file from storage system

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

flatMap() function maps line into words and retaining word if it is "spark".

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6}
"rdd.count()" will give the result 8.

Create an RDD by loading a file from storage system

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

flatMap() function maps line into words and filtering lines containing "spark".

Note not executed yet!

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD
"rdd.count()" will give the result 8.

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

flatMap() function maps line into words and filtering lines containing "spark".

Executed!

Apache Spark RDD Operations

RDD Action – some examples

- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD
"rdd.count()" will give the result 8.

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

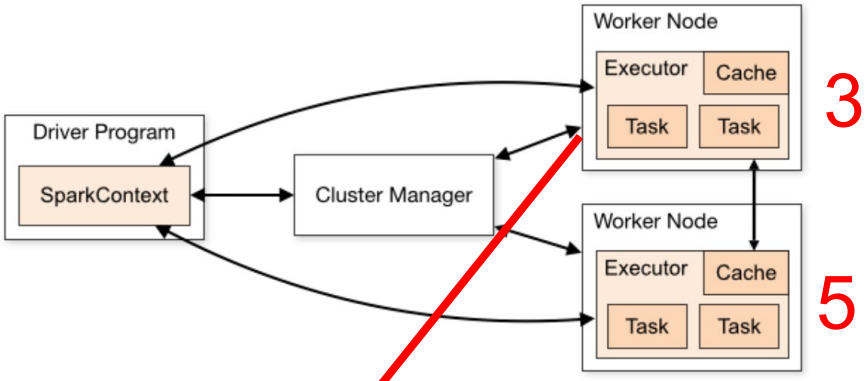
Count and print the number of elements in RDD mapFile

flatMap() function maps line into words and filtering lines containing "spark".

Executed!

Apache Spark RDD

RDD Action – some examples



- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD
"rdd.count()" will give the result 8.

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

Count and print the number of elements in RDD mapFile

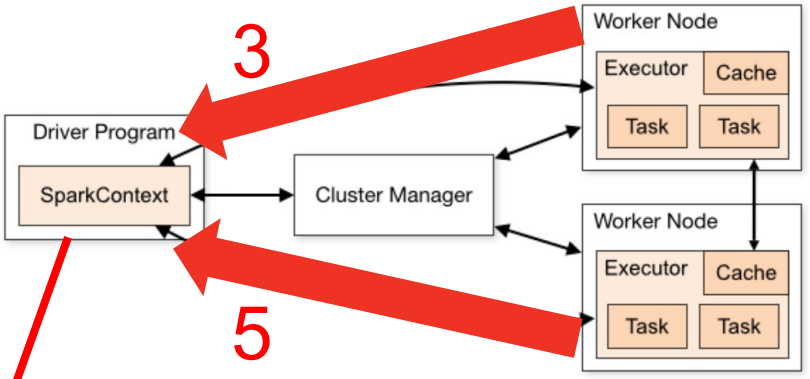
- Trigger transformations

flatMap() function maps line into words and filtering lines containing "spark".

Executed!

Apache Spark RDD

RDD Action – some examples



- count()

Action **count()** returns the number of elements in RDD.
For example, RDD has values {1, 2, 2, 3, 4, 5, 5, 6} in this RDD
"rdd.count()" will give the result 8.

```
val data = spark.read.textFile("spark_test.txt").rdd
val mapFile = data.flatMap(lines => lines.split(" ")).filter(value => value=="spark")
println(mapFile.count())
```

Count and print the number of elements in RDD mapFile

- Trigger transformations
- return to driver program

flatMap() function maps line into words and filtering lines containing "spark".

Executed!

Apache Spark RDD Operations

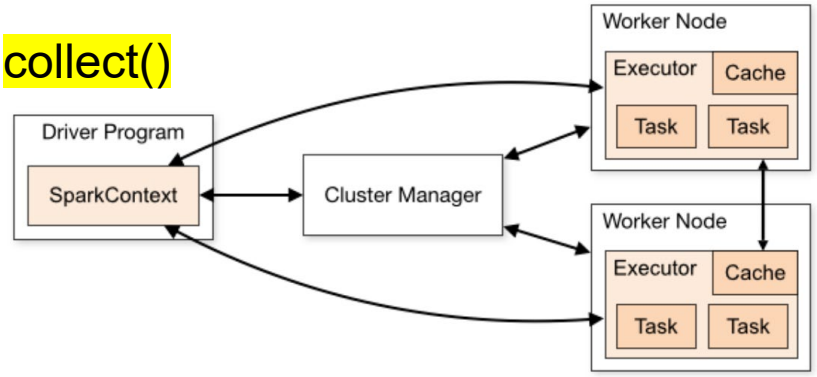
RDD Action – some examples

- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all of it's elements won't fit in memory on the drive machine, don't do this.

Apache Spark RDD Operations

RDD Action – some examples

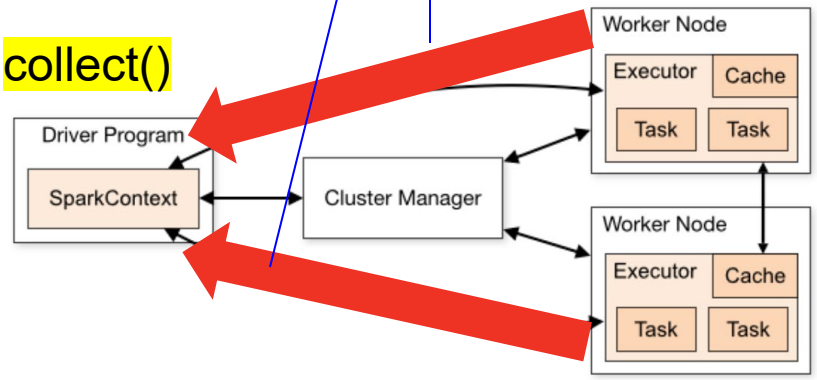
- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all of it's elements won't fit in memory on the drive machine, don't do this.



Apache Spark RDD Operations

RDD Action – some examples

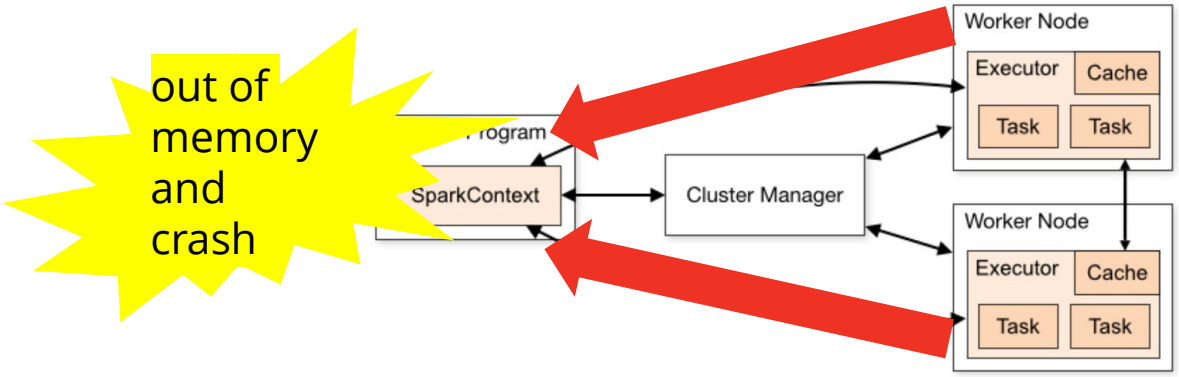
- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all of it's elements won't fit in memory on the drive machine, don't do this.



Apache Spark RDD Operations

RDD Action – some examples

- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all elements won't fit in memory on the drive machine, then run out of memory and crash. Don't do this.



Apache Spark RDD Operations

RDD Action – some examples

- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all elements won't fit in memory on the drive machine, then run out of memory and crash. Don't do this.

Instead, you can make sure the number of elements you return is capped by calling take or takeSample, or perhaps filtering or sampling your RDD.

Apache Spark RDD Operations

RDD Action – some examples

- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all elements won't fit in memory on the drive machine, then run out of memory and crash. Don't do this.

Instead, you can make sure the number of elements you return is capped by calling take or takeSample, or perhaps filtering or sampling your RDD.

```
val data = spark.sparkContext.parallelize(Array(('A',1),('b',2),('c',3)))
val data2 =spark.sparkContext.parallelize(Array(('A',4),('A',6),('b',7),('c',3),('c',8)))
```

(('A',1),('b',2),('c',3),('A',4),('A',6),('b',7),('c',3),('c',8)

Apache Spark RDD Operations

RDD Action – some examples

- collect() - returns entire RDDs content to driver program.
  - Collect will attempt to copy every single element in the RDD onto the single driver program
  - If your RDD is so large that all elements won't fit in memory on the drive machine, then run out of memory and crash. Don't do this.

Instead, you can make sure the number of elements you return is capped by calling take or takeSample, or perhaps filtering or sampling your RDD.

```
val data = spark.sparkContext.parallelize(Array(('A',1),('b',2),('c',3)))
val data2 =spark.sparkContext.parallelize(Array(('A',4),('A',6),('b',7),('c',3),('c',8)))
val result = data.join(data2)
println(result.collect())
```

('c',3)

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, “take (4)” will give result { 2, 2, 3, 4}

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first n number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, "take (4)" will give result { 2, 2, 3, 4}

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, “take (4)” will give result { 2, 2, 3, 4}

```
val data = spark.sparkContext.parallelize(Array(('k',5),('s',3),('s',4),('p',7),('t',8),('k',6)),3)
val group = data.groupByKey().collect()
val twoRec = group.take(2)
twoRec.foreach(println)
```

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, "take (4)" will give result { 2, 2, 3, 4}

```
val data = spark.sparkContext.parallelize(Array(('k',5),('s',3),('s',4),('p',7),('t',8),('k',6)),3)
val group = data.groupByKey().collect()
val twoRec = group.take(2)
twoRec.foreach(println)
```

(s, [3,4]), (p, [7]), (t, [8]), (k, [5.6])

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, "take (4)" will give result { 2, 2, 3, 4}

```
val data = spark.sparkContext.parallelize(Array(('k',5),('s',3),('s',4),('p',7),('t',8),('k',6)),3)
val group = data.groupByKey().collect()
val twoRec = group.take(2)
twoRec.foreach(println)
```

(s, [3,4]), (p, [7]), (t, [8]), (k, [5.6])

(s, [3,4]), (p, [7])

Apache Spark RDD Operations

RDD Action – some examples

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6}, "take (4)" will give result { 2, 2, 3, 4}

```
val data = spark.sparkContext.parallelize(Array(('k',5),('s',3),('s',4),('p',7),('t',8),('k',6)),3)
val group = data.groupByKey().collect()
val twoRec = group.take(2)
twoRec.foreach(println)
```

(s, [3,4])
(p, [7])

(s, [3,4]), (p, [7])

(s, [3,4]), (p, [7]), (t, [8]), (k, [5.6])

Normally, Spark tries to set the number of partitions automatically based on your cluster. However, you can also set it manually by passing it as a second parameter to parallelize

- take(n)

Returns first *n* number of elements from RDD. For example, consider RDD {2, 2, 3, 4, 5, 5, 6} in this RDD "take (4)" will give result { 2, 2, 3, 4}

```
val data = spark.sparkContext.parallelize(Array(('k',5),('s',3),('s',4),('p',7),('t',8),('k',6)),3)
val group = data.groupByKey().collect()
val twoRec = group.take(2)
twoRec.foreach(println)
```

Q2 - Transformations are lazy means that transformations execute only when we call an action. To save space of storage, can we modify the RDD directly by reading each line and then save the changed line back to the same RDD?

- ❑ True
- ❑ False

Q2 - Transformations are lazy means that transformations execute only when we call an action. To save space of storage, can we modify the RDD directly by reading each line and then save the changed line back to the same RDD?

- ❑ True
- ✓ False

- RDDs are immutable (read-only) in nature. You cannot change an original RDD,
- but you can create new RDDs by performing operations, like transformations, on an existing RDD.

Q3 - While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark. This will lead to DAG.

- ❑ True
- ❑ False

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - Transformations execute only when we call an action.
  - They are not executed immediately.
- RDD lineage
  - Applying transformation built an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as RDD operator graph or RDD dependency graph.

Apache Spark RDD Operations

Transformations are lazy

- Transformations are lazy in nature
  - When calling some transformation in RDD, it does not execute immediately. Spark maintains the record of which transformation is being called (Through DAG).

RMIT Classification: Trusted

Apache Spark RDD Operations

Transformations are lazy

- Transformations are lazy in nature
  - When calling some transformation in RDD, it does not execute immediately. Spark maintains the record of which transformation is being called (Through DAG).

RMIT Classification: Trusted

Apache Spark RDD Operations

Transformations are lazy

- Transformations are lazy in nature
  - When calling some transformation in RDD, it does not execute immediately. Spark maintains the record of which transformation is being called (Through DAG).
    - Since transformations are lazy in nature, so we can execute transformation any time by calling an action on data.
      - Hence, in lazy evaluation, data is not loaded until it is necessary.

Apache Spark RDD Operations

Transformations are lazy

- Transformations are lazy in nature
  - When calling some transformation in RDD, it does not execute immediately. Spark maintains the record of which transformation is being called (Through DAG).
    - Since transformations are lazy in nature, so we can execute transformation any time by calling an action on data.
      - Hence, in lazy evaluation, data is not loaded until it is necessary.

Directed Acyclic Graph (DAG)

Directed Acyclic Graph (DAG)

- DAG - a finite directed graph with no cycles.
  - Vertices represent the RDDs
  - Edges represent the transformation to be applied on RDD.
    - Every edge directs from earlier to later in the processing sequence.

Directed Acyclic Graph (DAG)

- DAG - a finite directed graph with no cycles.
  - Vertices represent the RDDs
  - Edges represent the transformation to be applied on RDD.
    - Every edge directs from earlier to later in the processing sequence.

Directed Acyclic Graph (DAG)

- DAG - a finite directed graph with no cycles.
  - Vertices represent the RDDs
  - Edges represent the transformation to be applied on RDD.
    - Every edge directs from earlier to later in the processing sequence.

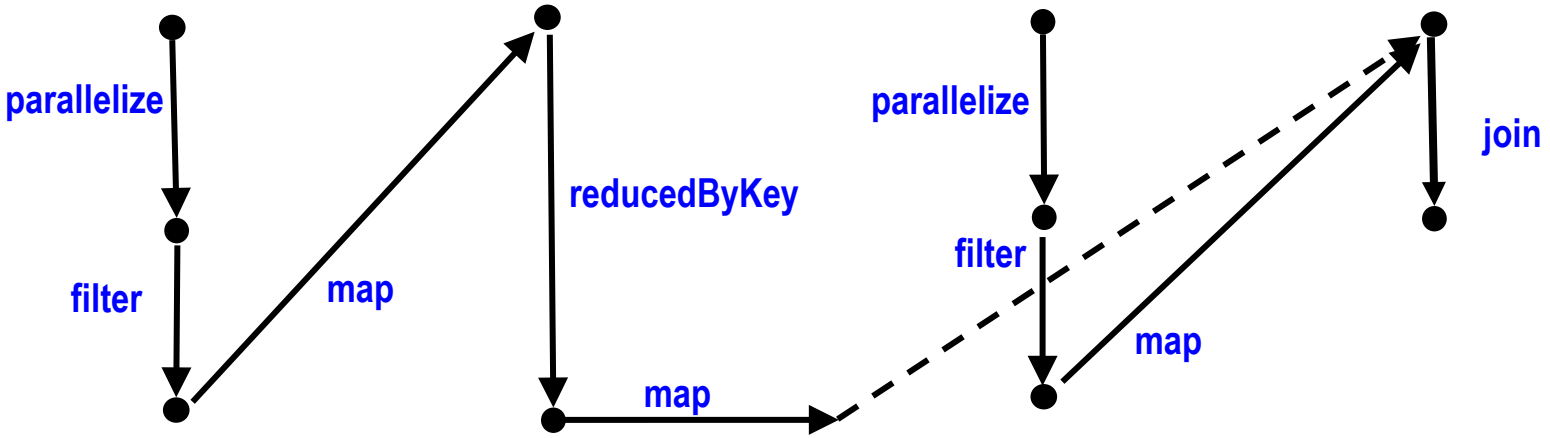
Directed Acyclic Graph (DAG)

- DAG - a finite directed graph with no cycles.
  - Vertices represent the RDDs
  - Edges represent the transformation to be applied on RDD.
    - Every edge directs from earlier to later in the processing sequence.
- On the calling of Action, the created DAG submits to DAG Scheduler
  - DAG Scheduler splits the graph into the stages.

Directed Acyclic Graph (DAG)

- DAG - a finite directed graph with no cycles.
  - Vertices represent the RDDs
  - Edges represent the transformation to be applied on RDD.
    - Every edge directs from earlier to later in the processing sequence.
- On the calling of Action, the created DAG submits to DAG Scheduler
  - DAG Scheduler splits the graph into the stages.

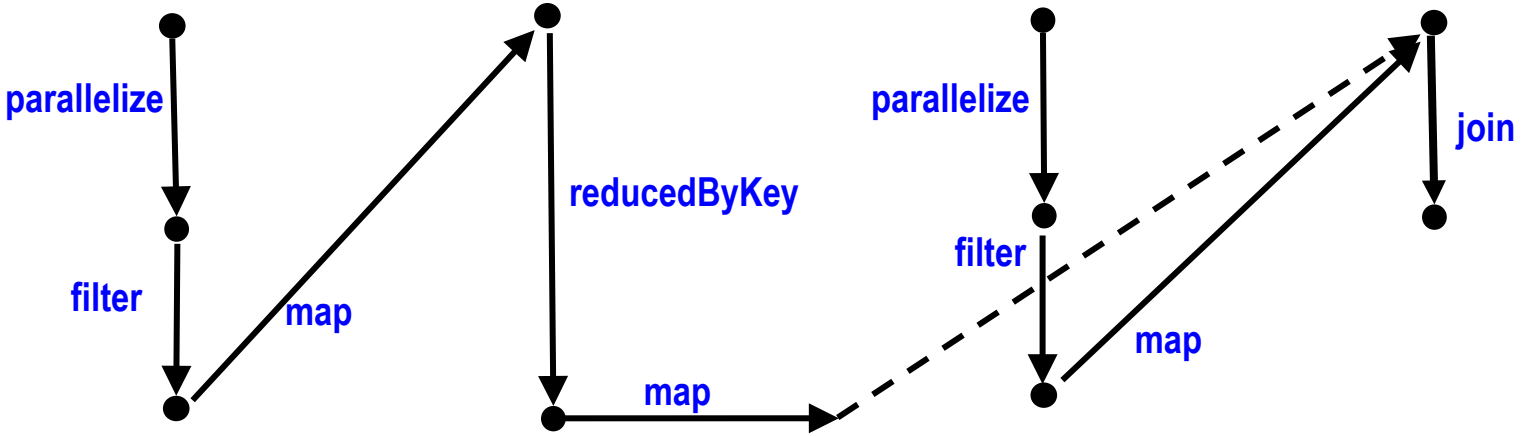
Directed Acyclic Graph (DAG) - An Example



- RDD → Edge - the transformation from earlier to later in the processing sequence

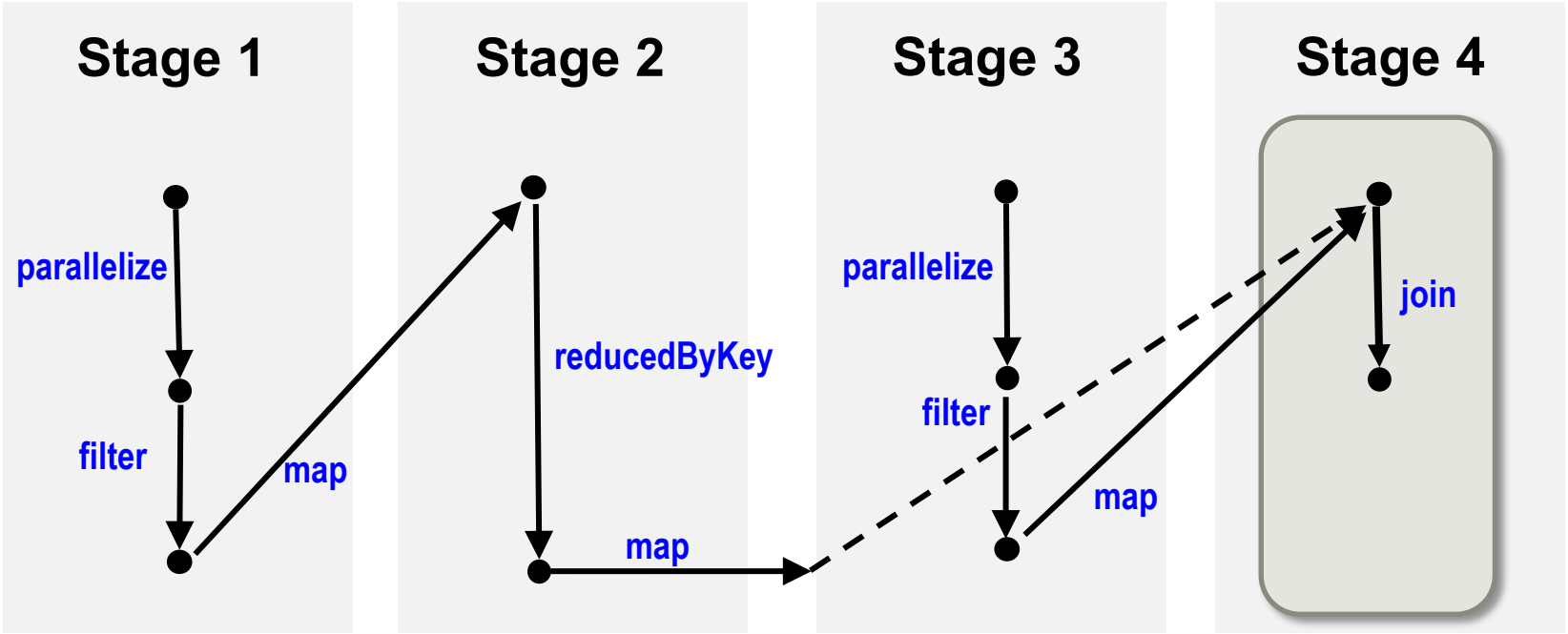
Directed Acyclic Graph (DAG) - An Example

- On the calling of Action, the created DAG submits to DAG Scheduler



Directed Acyclic Graph (DAG) - An Example

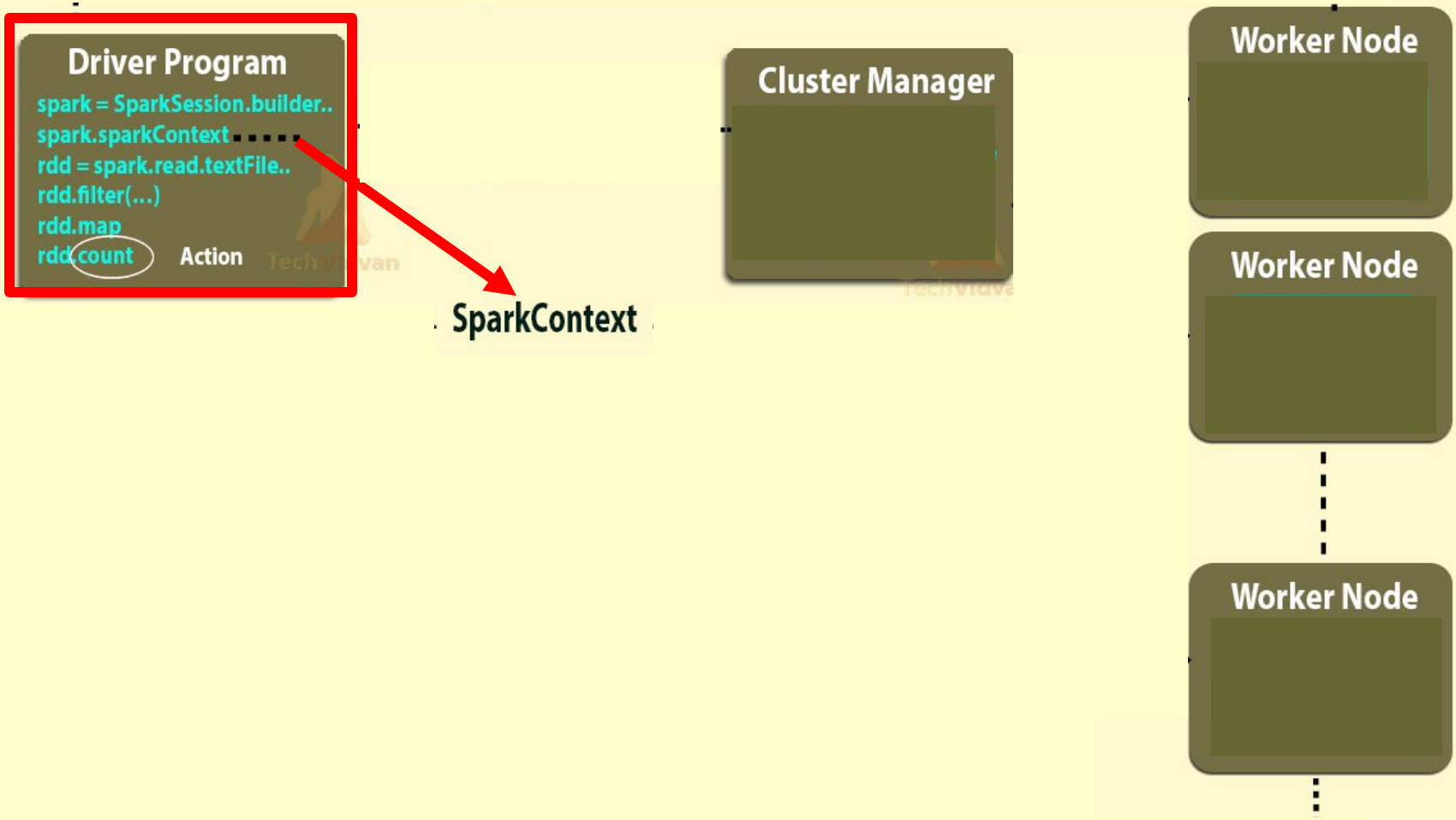
- On the calling of Action, the created DAG submits to DAG Scheduler
- DAG Scheduler splits the graph into the stages.



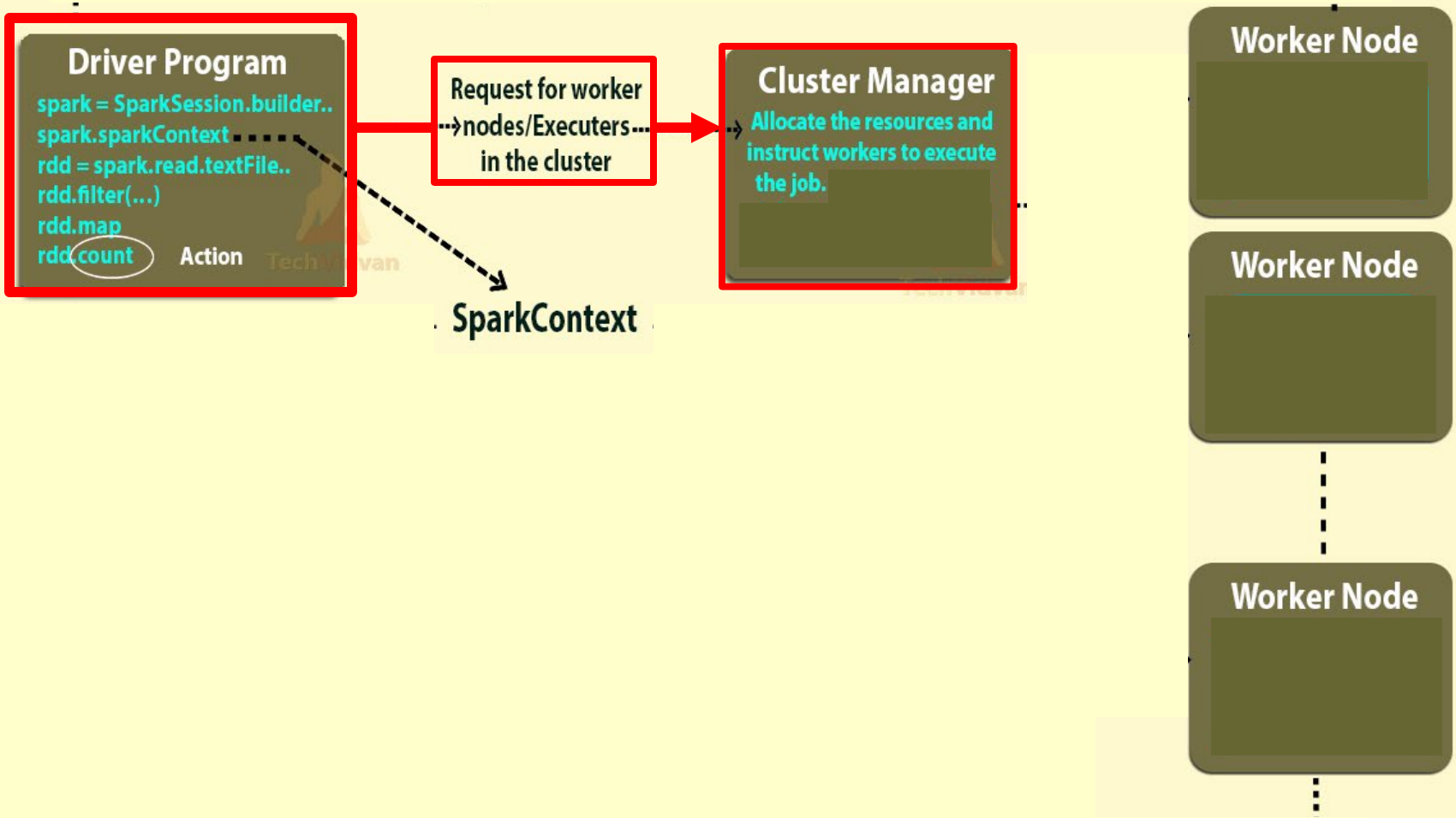
Internals of Job Execution In Spark



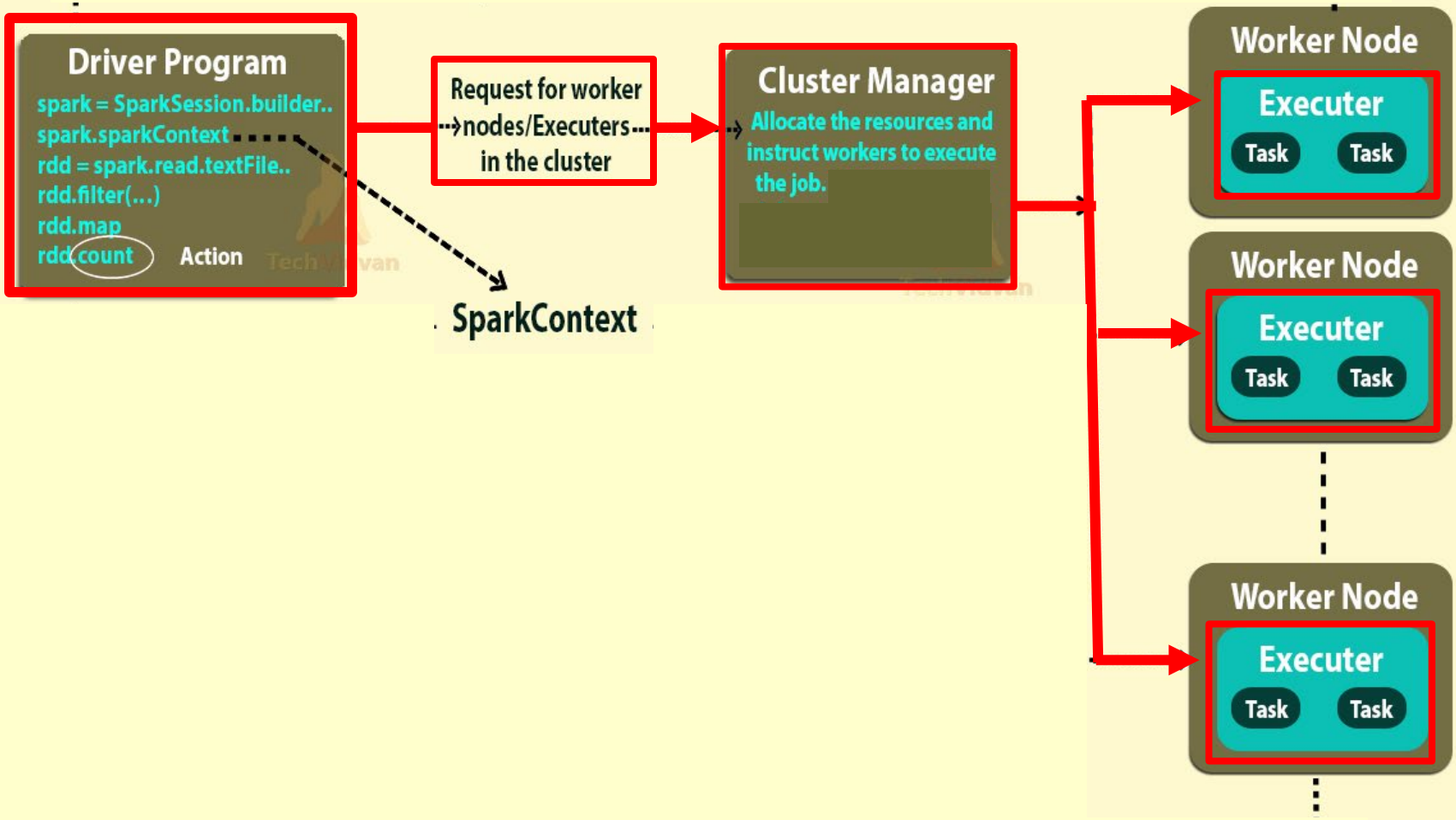
Internals of Job Execution In Spark



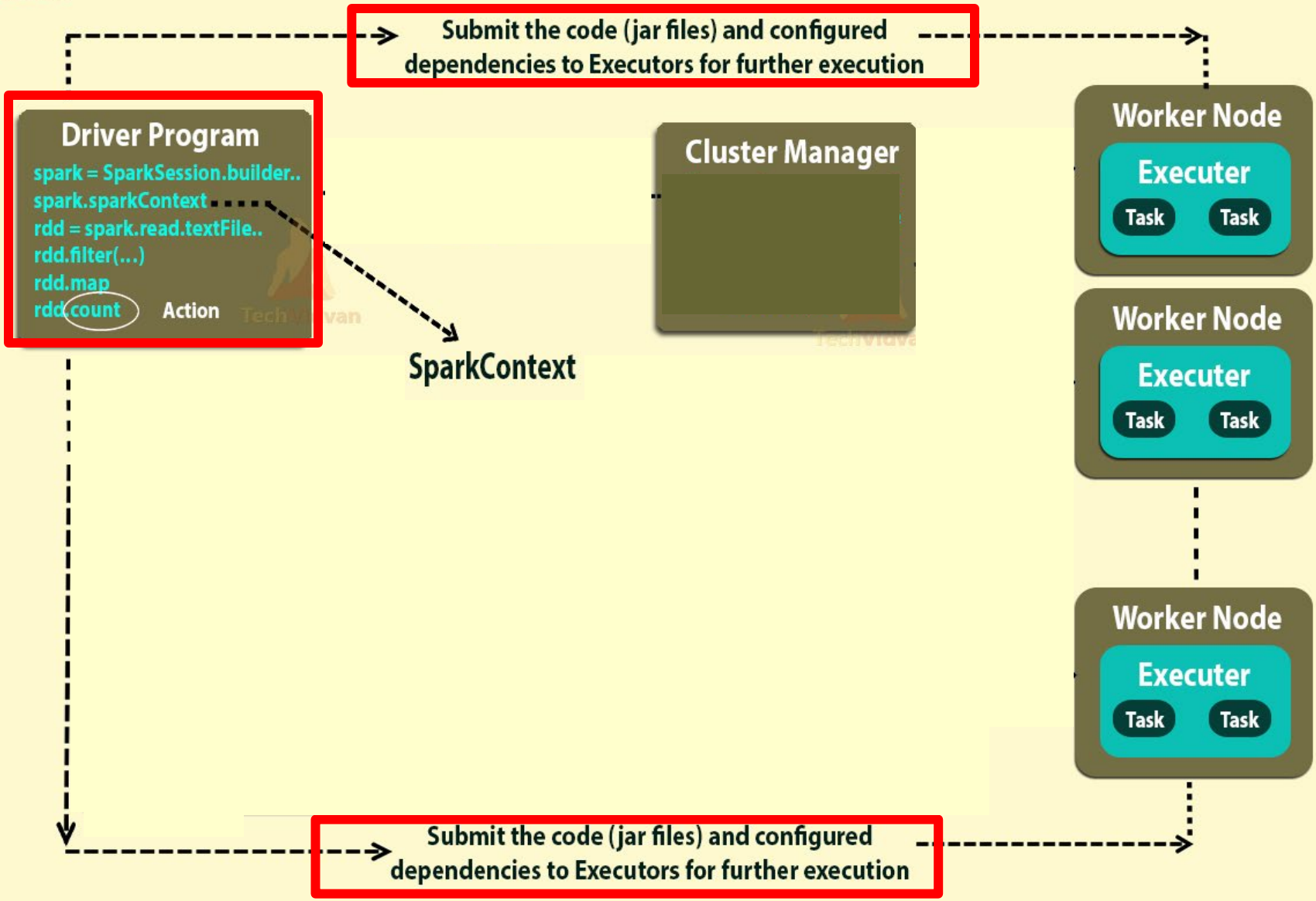
Internals of Job Execution In Spark



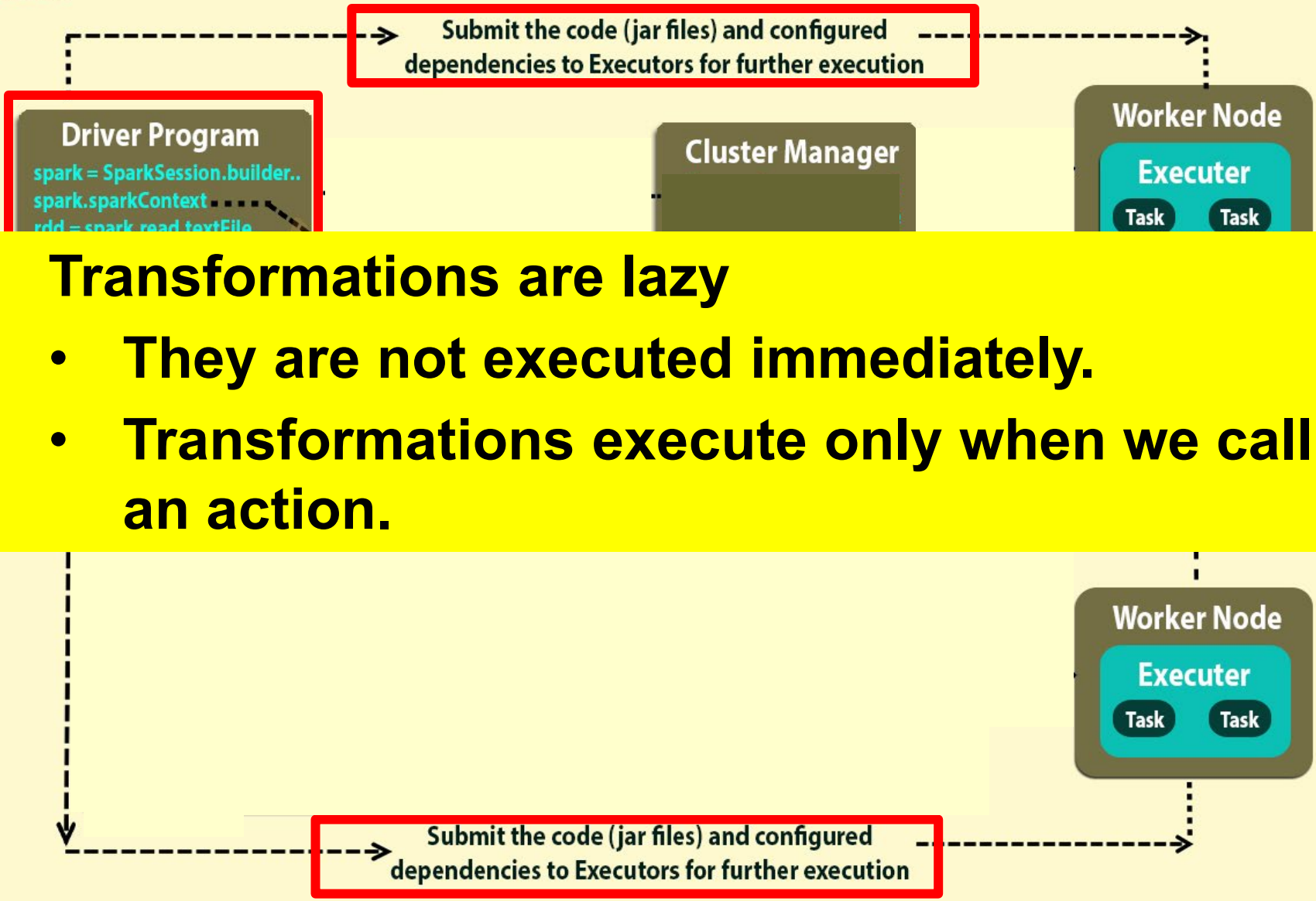
Internals of Job Execution In Spark



Internals of Job Execution In Spark



Internals of Job Execution In Spark

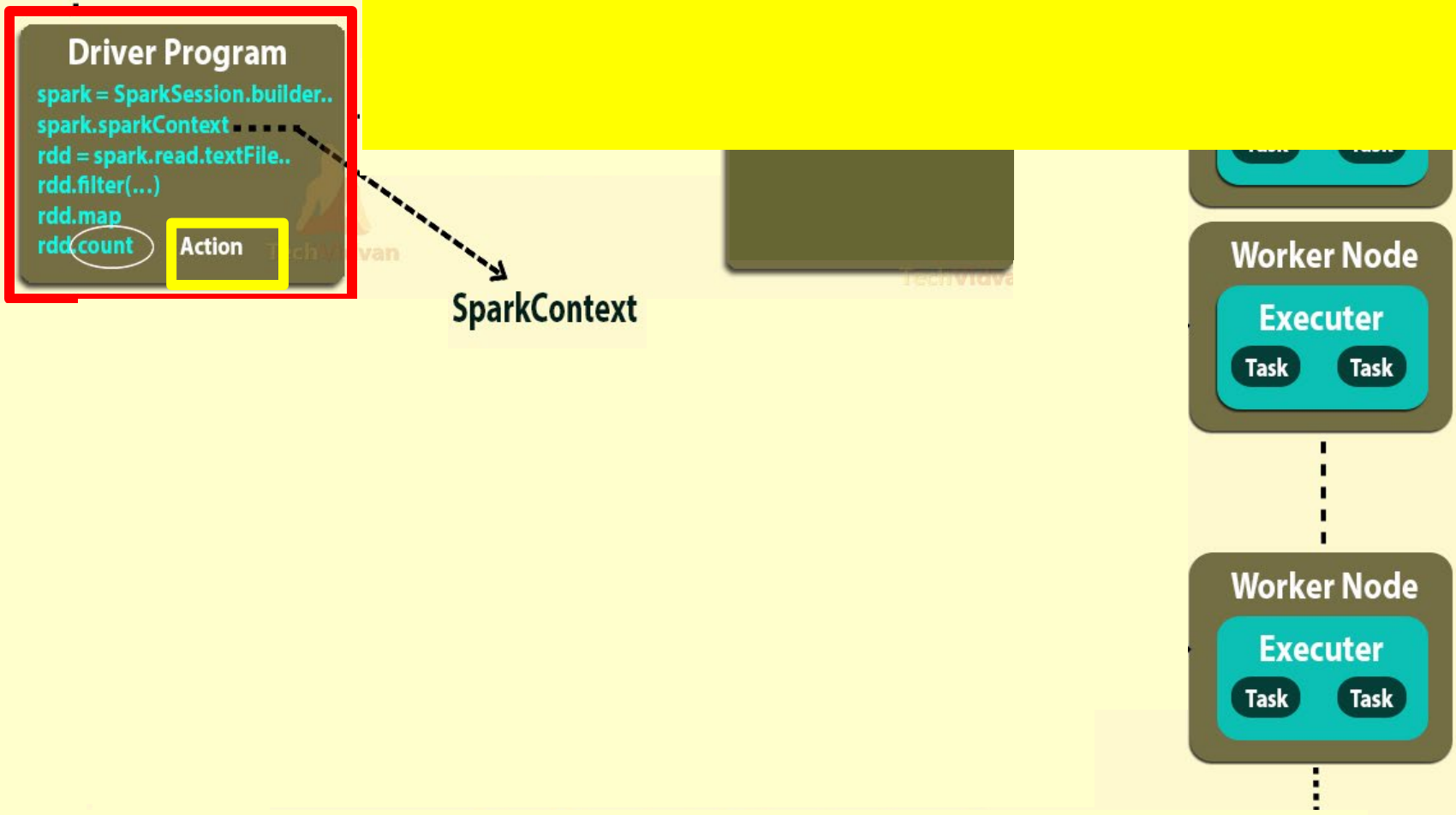


TechVidvan

Intern

On the calling of Action

such as count(), foreachRdd(), collect(), read() or write(),

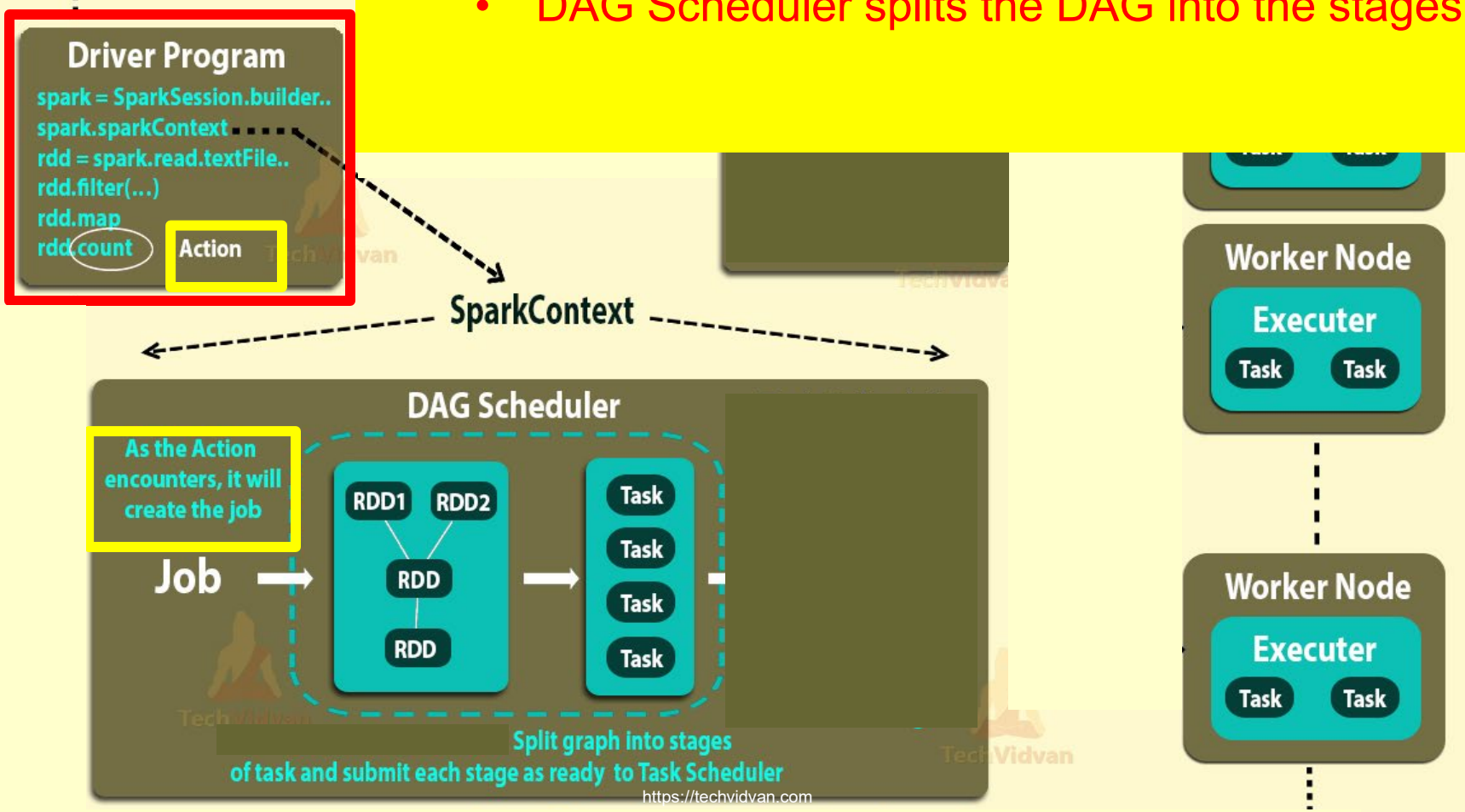


Intern

On the calling of Action

such as count(), foreachRdd(), collect(), read() or write(),

- **The created DAG submits to DAG Scheduler**
  - DAG Scheduler splits the DAG into the stages

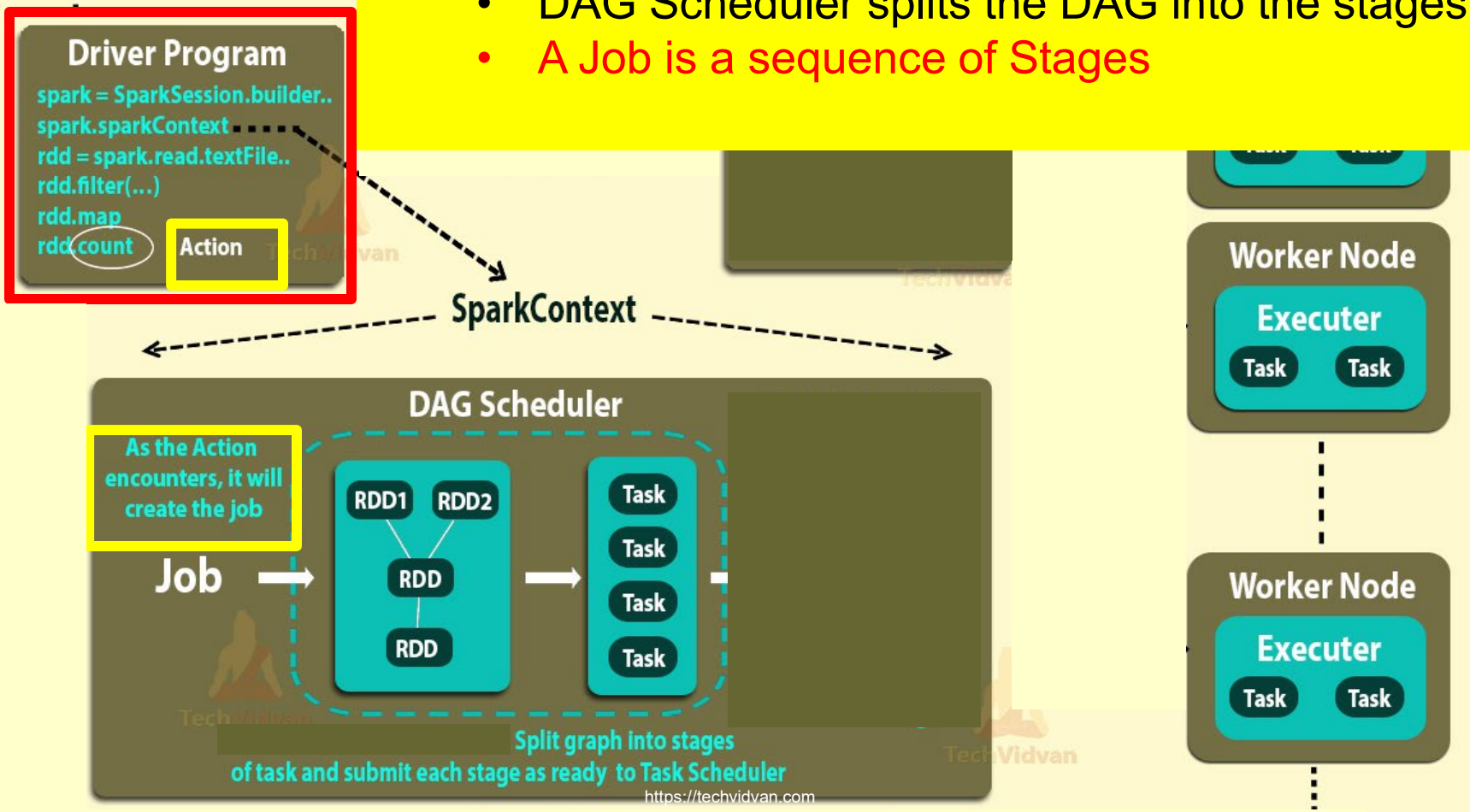


TechVidvan
https://techvidvan.com

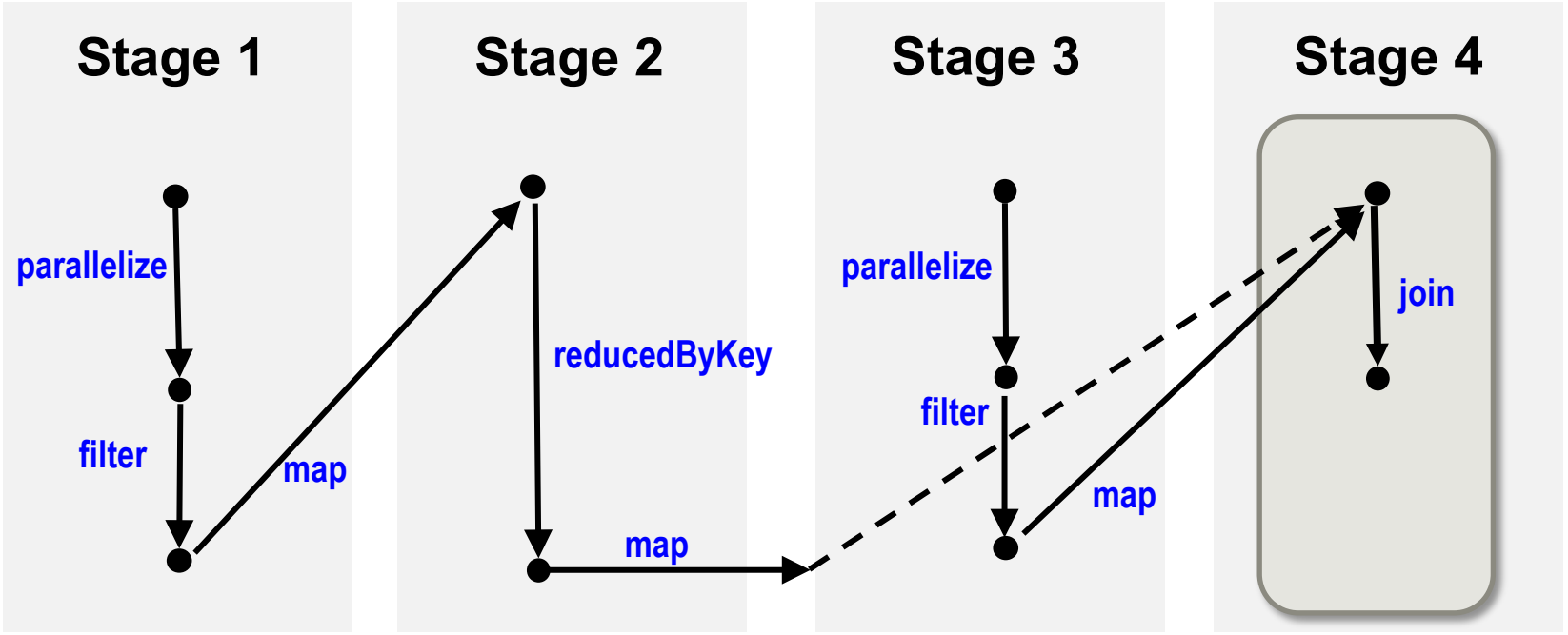
On the calling of Action

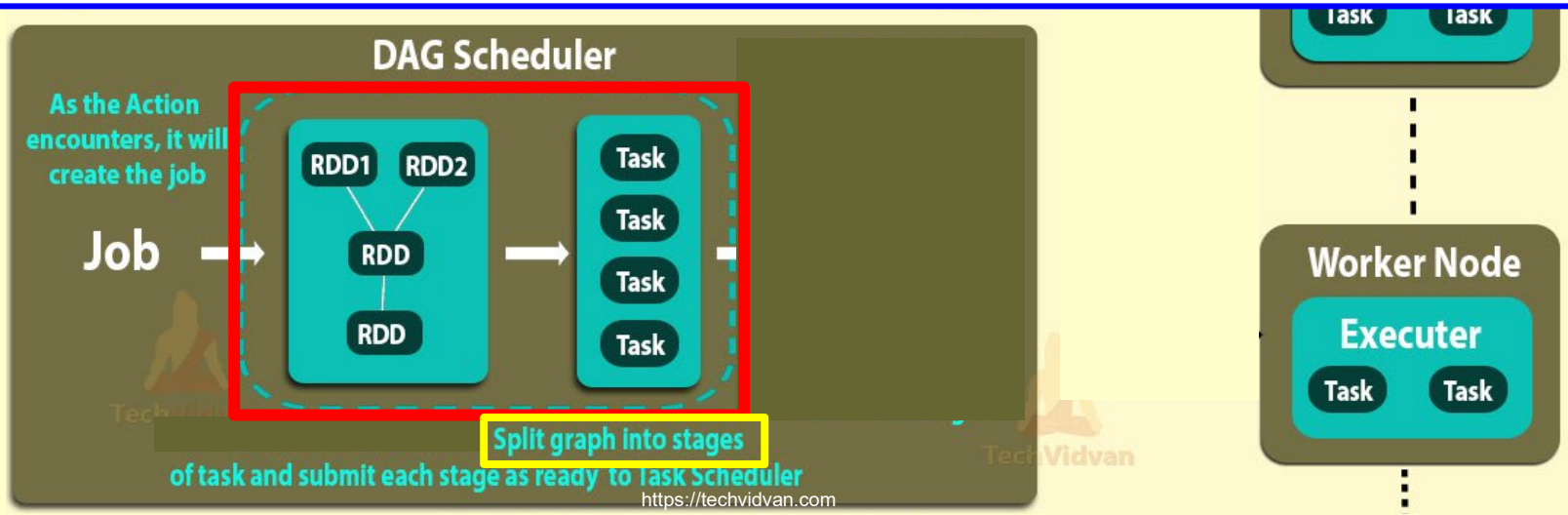
such as count(), foreachRdd(), collect(), read() or write(),

- **The created DAG submits to DAG Scheduler**
  - DAG Scheduler splits the DAG into the stages
  - A Job is a sequence of Stages

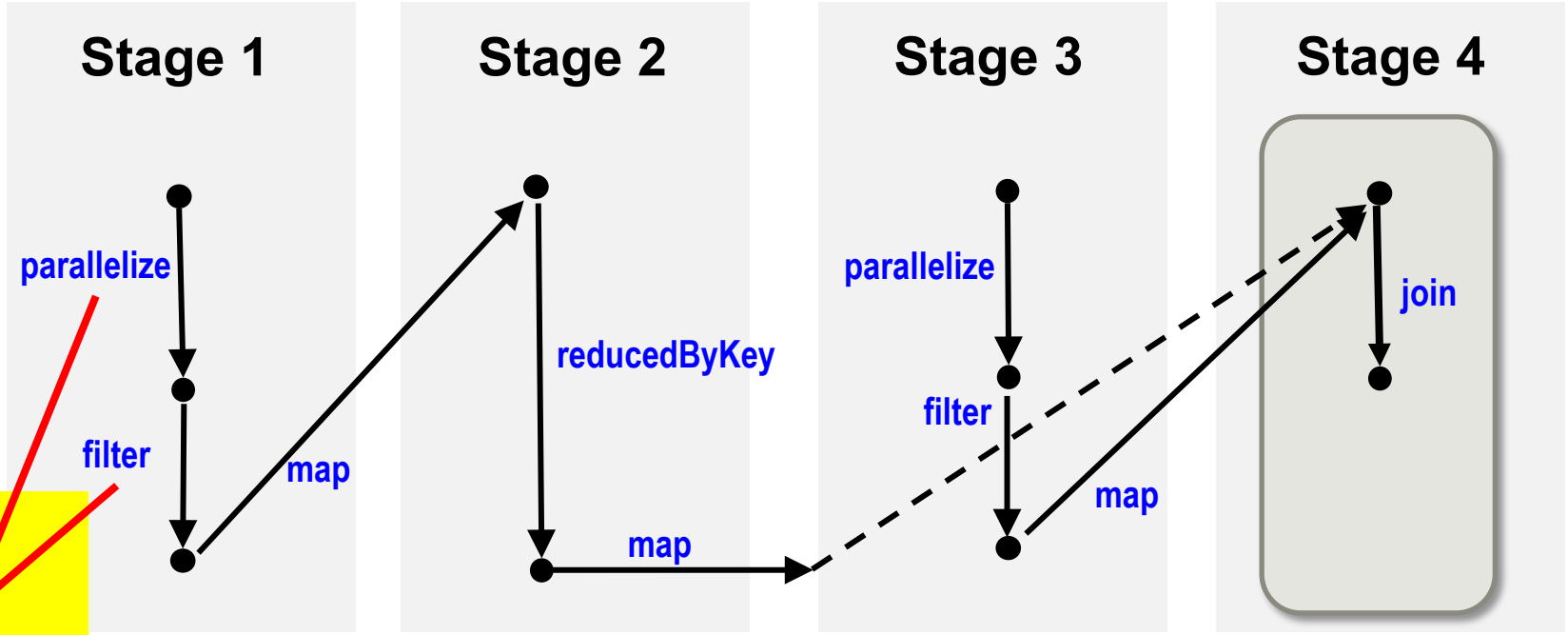


A job

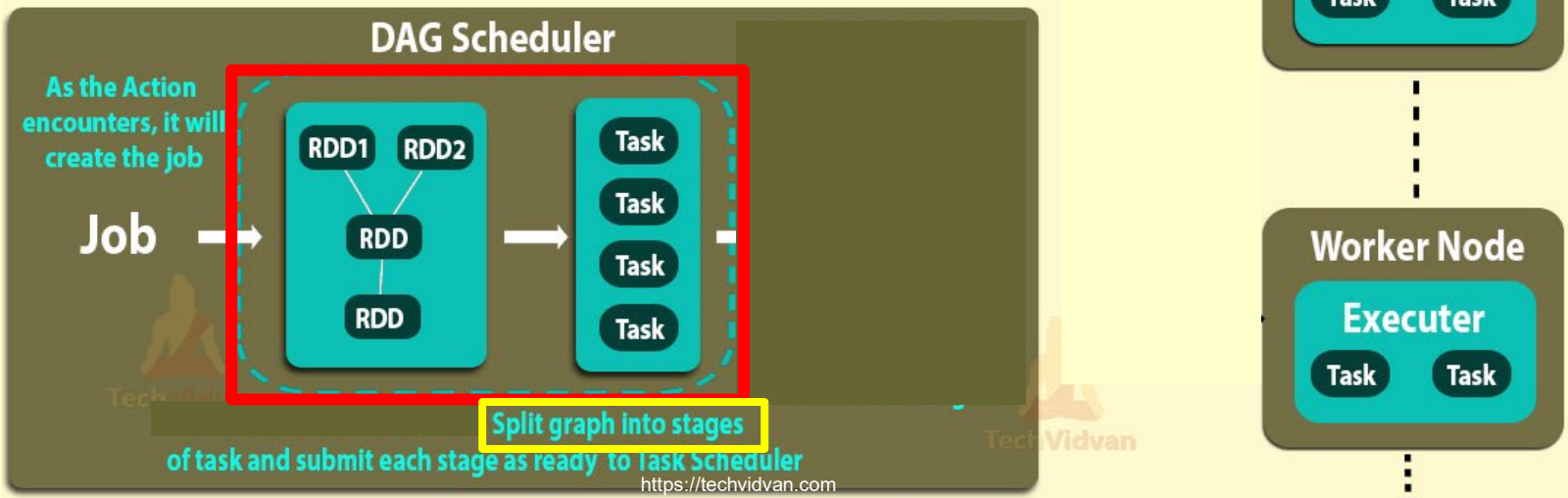

Stage 1
Stage 2
Stage 3
Stage 4
parallelize
filter
map
reducedByKey
map
parallelize
filter
map
join


DAG Scheduler
As the Action encounters, it will create the job
Job
RDD1
RDD2
RDD
RDD
Task
Task
Task
Task
Split graph into stages
of task and submit each stage as ready to Task Scheduler
https://techvidvan.com
Worker Node
Executer
Task
Task

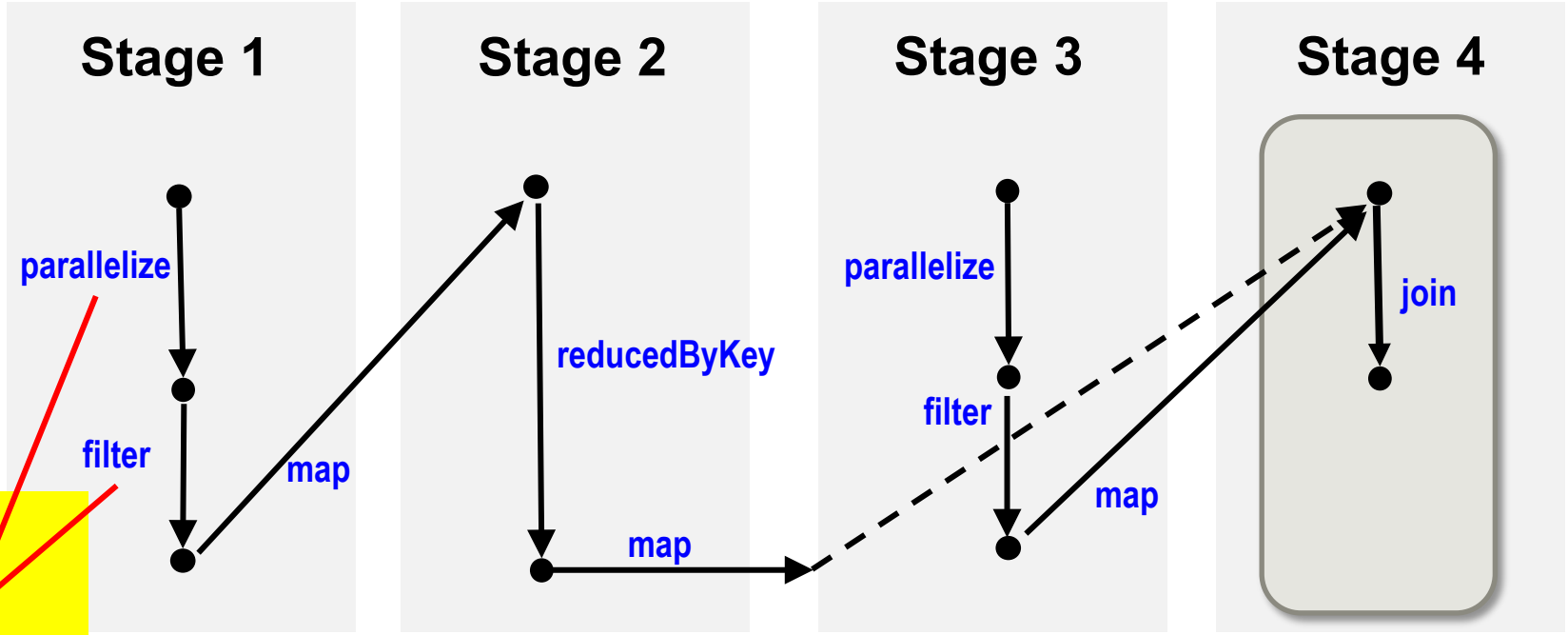
A job



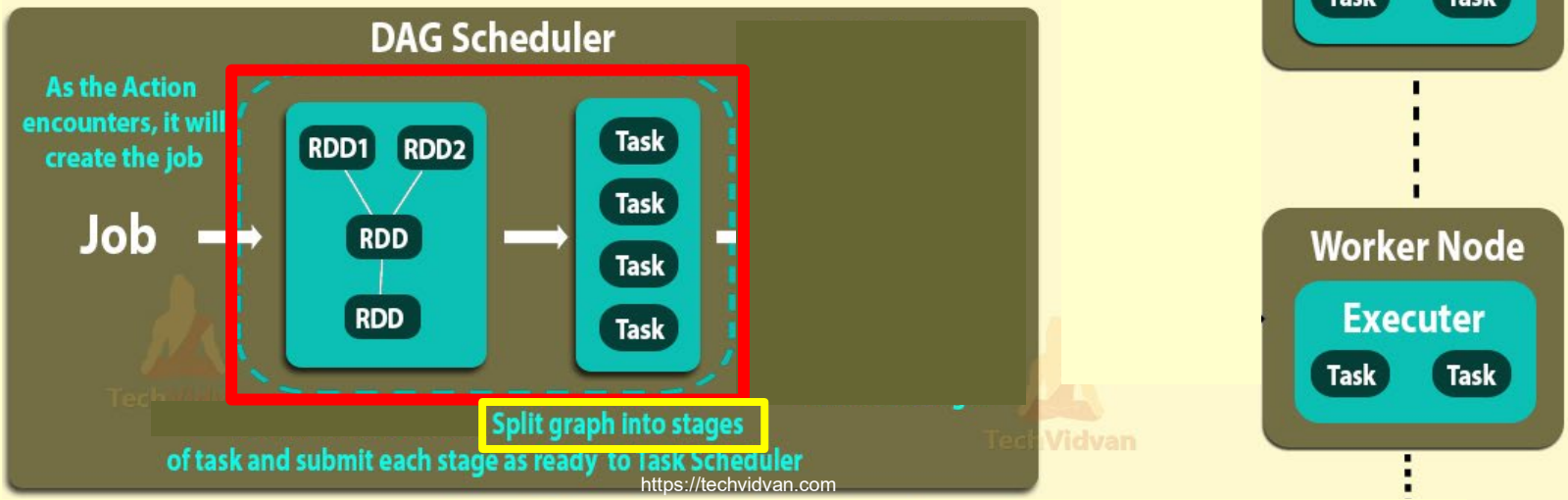
A Stage is a sequence of Tasks.



A job

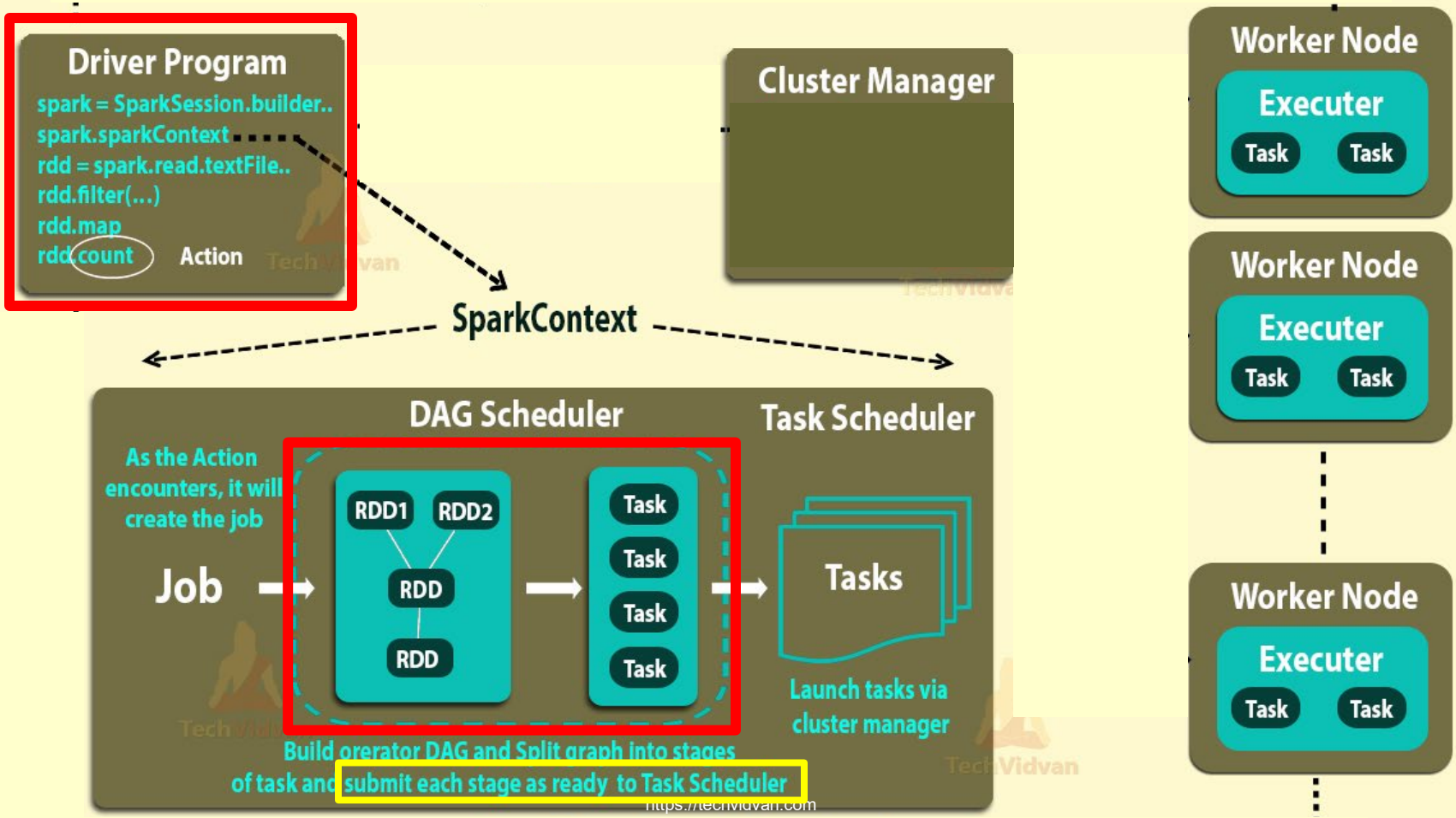


A Stage is a sequence of Tasks.

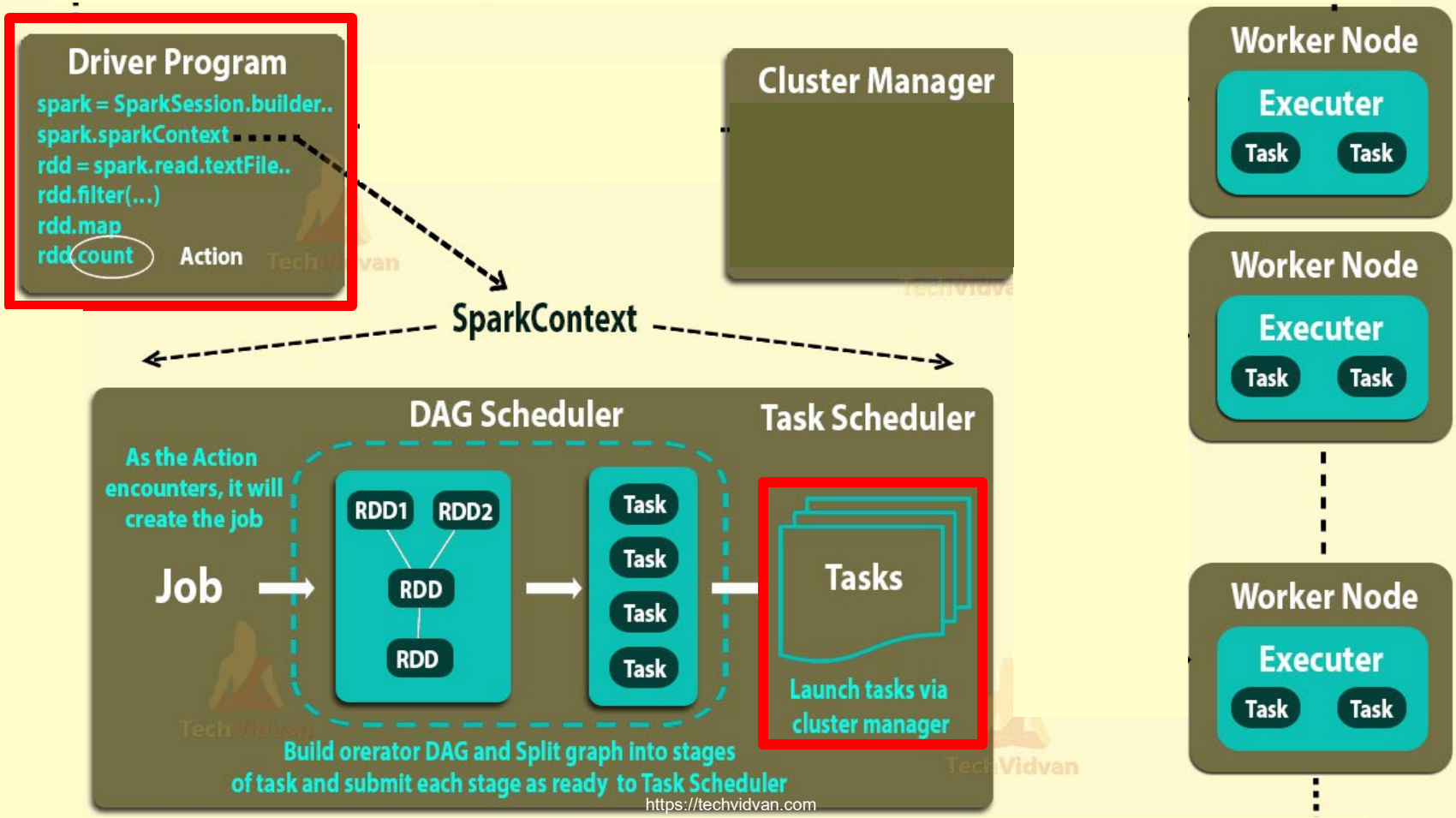


For *narrow transformation task*, for example *filter*, if RDD is split into 3 partitions, there are 3 filtering tasks

Internals of Job Execution In Spark

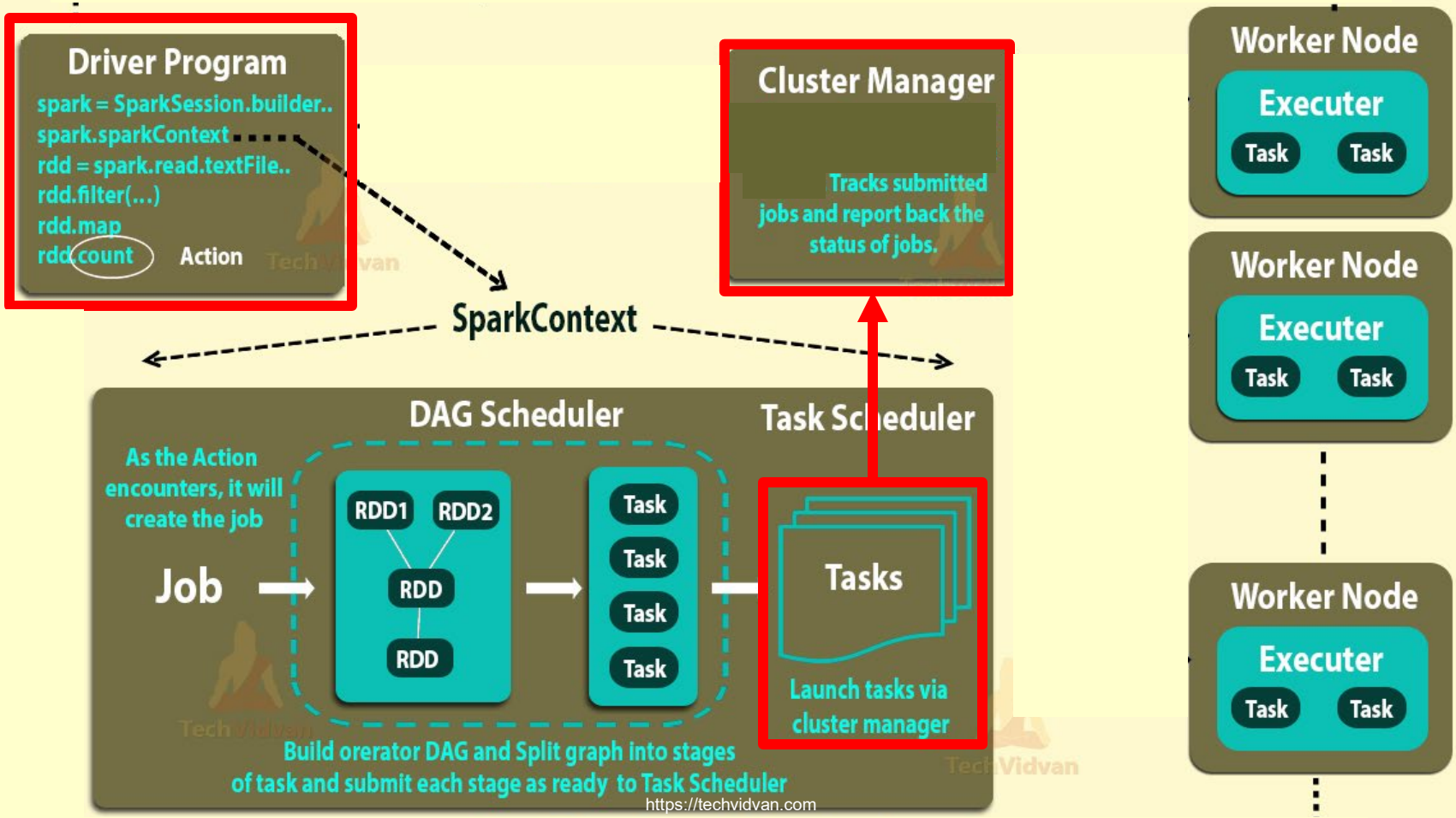


Internals of Job Execution In Spark



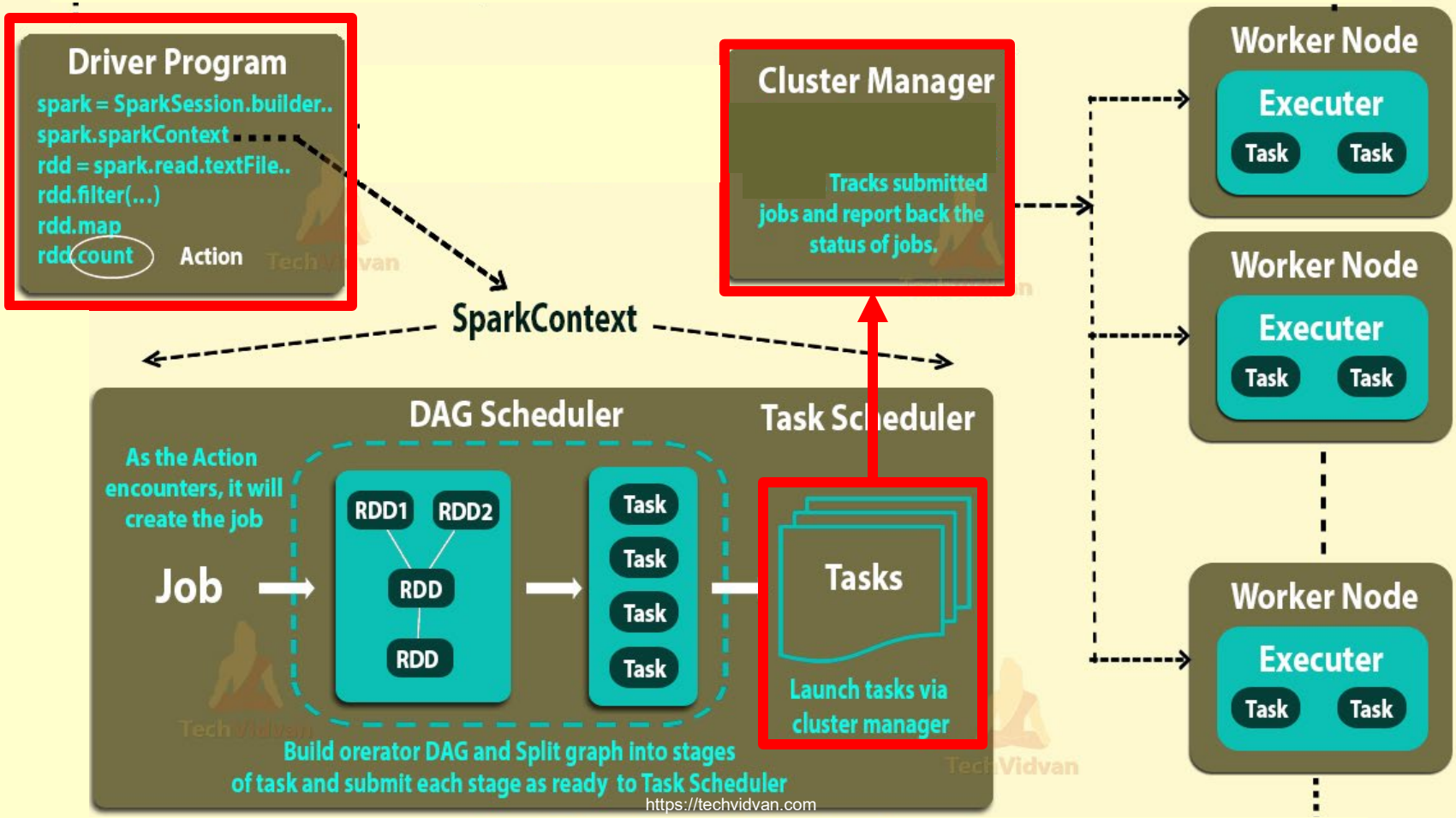
https://techvidvan.com

Internals of Job Execution In Spark



https://techvidvan.com

Internals of Job Execution In Spark



Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs cannot be changed since RDD are immutable in nature.
- Transformations are lazy
  - Transformations get execute when we call an action.
  - They are not executed immediately.
- RDD lineage
  - "Applying transformation" builts an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as
    - RDD operator graph
    - RDD dependency graph
    - Operator DAG

Apache Spark RDD Operations

RDD Transformation

- Spark Transformation is a function
  - It takes existing RDDs as input and produces one or more RDD as output.
    - The input RDDs are changed since RDD are immutable in nature.
- Transformations are lazy
  - Transformations get execute when we call an action.
  - They are not executed immediately.
- RDD lineage
  - "Applying transformation" builts an RDD lineage, with the entire parent RDDs of the final RDD(s).
  - RDD lineage, also known as
    - RDD operator graph
    - RDD dependency graph
    - Operator DAG

Apache Spark RDD Operations

RDD Lineage

- While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark.
  - All the dependencies between the RDDs are logged in a graph, rather than the actual data. It is why we call as lineage graph.

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Lineage

- While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark.
  - All the dependencies between the RDDs are logged in a graph, rather than the actual data. It is why we call as lineage graph.

Apache Spark RDD Operations

RDD Lineage

- While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark.
  - All the dependencies between the RDDs are logged in a graph, rather than the actual data. It is why we call as lineage graph.

For example:

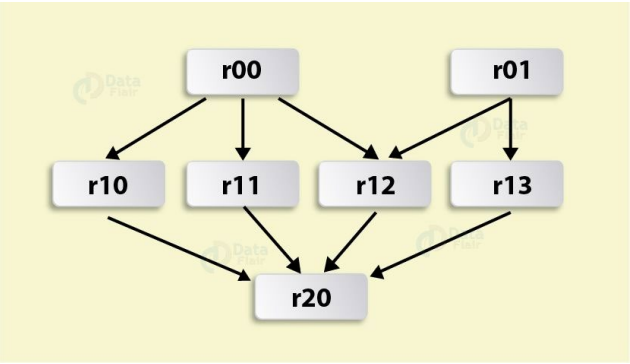
```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

RMIT Classification: Trusted

Apache Spark RDD Operations

RDD Lineage

- While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark.
  - All the dependencies between the RDDs are logged in a graph, rather than the actual data. It is why we call as lineage graph.



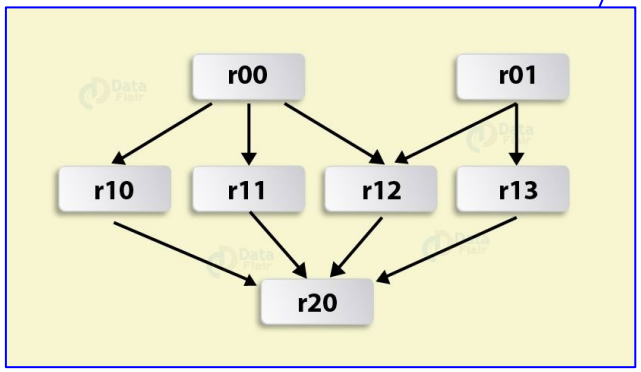
For example:

```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Apache Spark RDD Operations

RDD Lineage vs. DAG

DAG includes all stages of a spark job



For example:

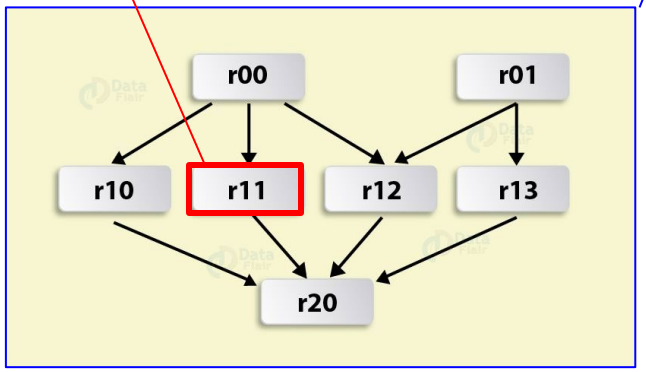
```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Apache Spark RDD Operations

RDD Lineage vs. DAG

Lineage deals with RDDs related to transformations **applied.**

DAG includes all stages of a spark job



For example:

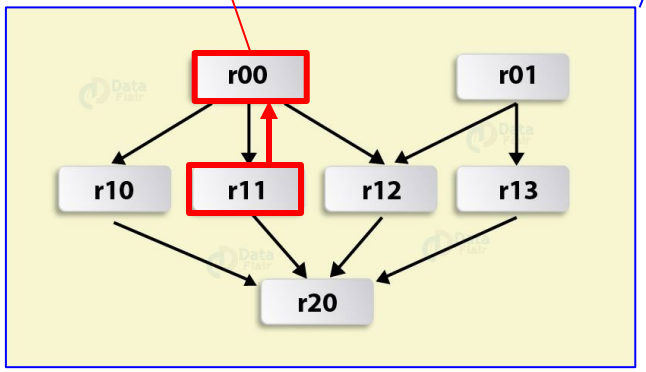
```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Apache Spark RDD Operations

RDD Lineage vs. DAG

Lineage deals with RDDs related to transformations **applied.**

DAG includes all stages of a spark job, i.e., complete task (both transformation and Action).



For example:

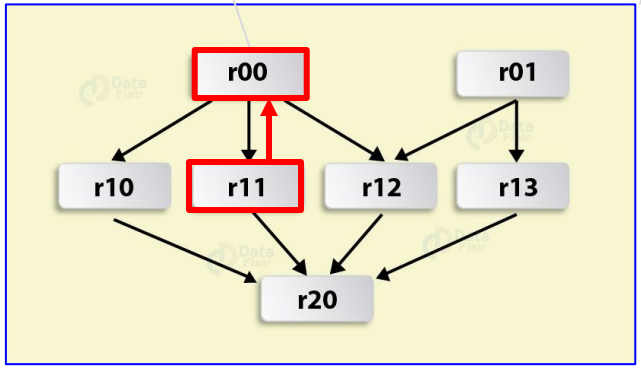
```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Apache Spark RDD Operations

RDD Lineage vs. DAG

Lineage deals with RDDs related to transformations **applied.**

DAG includes all stages of a spark job, i.e., complete task (both transformation and Action).



For example:

```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Whenever RDD r11 is lost, r11 can be recovered using the lineage graph information.

Q3 - While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark. This will lead to DAG.

- ❑ True
- ❑ False

Q3 - While we create a new RDD from an existing Spark RDD, that new RDD also carries a pointer to the parent RDD in Spark. This will lead to DAG.

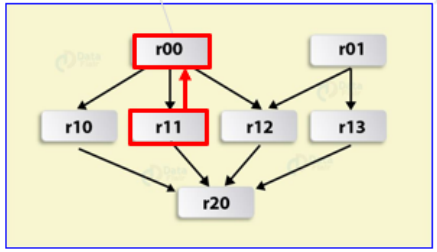
- ❑ True
- ✓ False

Apache Spark RDD Operations

RDD Lineage vs. DAG

Lineage deals with RDDs related to transformations **applied.**

DAG includes all stages of a spark job, i.e., complete task (both transformation and Action).



For example:

```
val r00 = sc.parallelize(0 to 9)
val r01 = sc.parallelize(0 to 90 by 10)
val r10 = r00 cartesian df01
val r11 = r00.map(n => (n, n))
val r12 = r00 zip df01
val r13 = r01.keyBy(_ / 20)
val r20 = Seq(r11, r12, r13).foldLeft(r10)(_ union _)
```

Whenever RDD r11 is lost, r11 can be recovered using the lineage graph information.

Q4 - When calling some transformation, it does not execute immediately. Spark maintains the record of which transformation is being called via DAG. Which of the following is correct?

- ❑ DAG scheduler is responsible for task scheduling.
- ❑ DAG is used to recover the RDD, lost or damaged due to system failure.
- ❑ DAG is sent to the executor on each worker node for transforming RDD.
- ❑ DAG and RDD linage have different purposes even though they are related in some way.

Q4 - When calling some transformation, it does not execute immediately. Spark maintains the record of which transformation is being called via DAG. Which of the following is correct?



- DAG scheduler is responsible for task scheduling.
- DAG is used to recover the RDD, lost or damaged due to system failure.
- DAG is sent to the executor on each worker node for transforming RDD.
- DAG and RDD linage have different purposes even though they are related in some way.

Q4 - When calling some transformation, it does not execute immediately. Spark maintains the record of which transformation is being called via DAG. Which of the following is correct?



DAG scheduler is responsible for task scheduling.



DAG is used to recover the RDD, lost or damaged due to system failure.

- ❑ DAG is sent to the executor on each worker node for transforming RDD.
- ❑ DAG and RDD linage have different purposes even though they are related in some way.

Q4 - When calling some transformation, it does not execute immediately. Spark maintains the record of which transformation is being called via DAG. Which of the following is correct?



DAG scheduler is responsible for task scheduling.



DAG is used to recover the RDD, lost or damaged due to system failure.



DAG is sent to the executor on each worker node for transforming RDD.

- ❑ DAG and RDD linage have different purposes even though they are related in some way.

Q4 - When calling some transformation, it does not execute immediately. Spark maintains the record of which transformation is being called via DAG. Which of the following is correct?



DAG scheduler is responsible for task scheduling.



DAG is used to recover the RDD, lost or damaged due to system failure.



DAG is sent to the executor on each worker node for transforming RDD.



DAG and RDD linage have different purposes even though they are related in some way.

Q5 - RDD is the acroym of Resilient Distributed Dataset. If an RDD is damaged or lost, we should be able to recover it. Which of the followings is the mechanism of Spark to do so?

- Maintain the RDD lineage
- Pick the corresponding stage in DAG execution plan and send tasks to executors for recovery
- Since transformations are lazy, so do nothing only when some action is called.
- Redo the transformation on input RDD against DAG

Q5 - RDD is the acroym of Resilient Distributed Dataset. If an RDD is damaged or lost, we should be able to recover it. Which of the followings is the mechanism of Spark to do so?



Maintain the RDD lineage

- ❑ Pick the corresponding stage in DAG execution plan and send tasks to executors for recovery
- ❑ Since transformations are lazy, so do nothing only when some action is called.
- ❑ Redo the transformation on input RDD against DAG

Q5 - RDD is the acroym of Resilient Distributed Dataset. If an RDD is damaged or lost, we should be able to recover it. Which of the followings is the mechanism of Spark to do so?



Maintain the RDD lineage



Pick the corresponding stage in DAG execution plan and send tasks to executors for recovery

- ❑ Since transformations are lazy, so do nothing only when some action is called.
- ❑ Redo the transformation on input RDD against DAG

Q5 - RDD is the acroym of Resilient Distributed Dataset. If an RDD is damaged or lost, we should be able to recover it. Which of the followings is the mechanism of Spark to do so?



Maintain the RDD lineage



Pick the corresponding stage in DAG exectuion plan and send tasks to executors for recovery



Since transformations are lazy, so do nothing only when some action is called.

❑ Redo the transformation on input RDD against DAG

Q5 - RDD is the acroym of Resilient Distributed Dataset. If an RDD is damaged or lost, we should be able to recover it. Which of the followings is the mechanism of Spark to do so?



Maintain the RDD lineage



Pick the corresponding stage in DAG exectuion plan and send tasks to executors for recovery



Since transformations are lazy, so do nothing only when some action is called.



Redo the transformation on input RDD against DAG

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.

- The main function of driver program will fall in infinite loop
- The transformations will be executed in different worker nodes
- Crash system due to out of memory
- It is extremely slow

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.



- The main function of driver program will fall in infinite loop
- The transformations will be executed in different worker nodes
- Crash system due to out of memory
- It is extremely slow

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.



The main function of driver program will fall in infinite loop



The transformations will be executed in different worker nodes

- ❑ Crash system due to out of memory
- ❑ It is extremely slow

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.



The main function of driver program will fall in infinite loop



The transformations will be executed in different worker nodes



Crash system due to out of memory

- ❑ It is extremely slow

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.



The main function of driver program will fall in infinite loop



The transformations will be executed in different worker nodes



Crash system due to out of memory



It is extremely slow

Q6 - "collect" is an action which returns RDD to driver program. What is the potential risk by calling this action.

- The main function of driver program will fall in infinite loop
- The transformations will be executed in different worker nodes
- Crash system due to out of memory
- It is extremely slow

Questions?